SENTIMENTAL ANALYSIS USING SOCIAL MEDIA COMMENTS

A PROJECT REPORT

Submitted by

N SATYA DINESH [RA2211003010795]

AMRITANSHU SHIWANSHI [RA2211003010756]

Under the Guidance of

Dr. M. Murali

Professor, Department of Computing Technologies

in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTING TECHNOLOGIES

COLLEGE OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR – 603 203

MAY 2025

Department of Computing Technologies
SRM Institute of Science & Technology
Own Work Declaration Form

This sheet must be filled in (each box ticked to show that the condition has been met). It must be signed and dated along with your student registration number and included with all assignments you submit – work will not be marked unless this is done.

To be completed by the student for all assessments

Degree/Course: BTECH COMPUTER SCIENCE AND ENGINEERING

Student Name: AMRITANSHU SHIWANSHI & N SATYA DINESH

Registration Number: RA2211003010756 & RA2211003010795

Title of the Work: Sentimental Analysis Using Social Media Comments

I / We hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

I / We confirm that all the work contained in this assessment is my / our own except where indicated, and that I / We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g.fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

I understand that any false claim for this work will be penalized in accordance with the Universities policies and regulations.

DECLARATION:
I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my / our own work, except were indicated by referring, and that I have followed the good academic practices noted above.
N SATYA DINESH (RA2211003010795) AMRITANSHU SHIWANSHI (RA2211003010756)

ACKNOWLEDGEMENT

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M,** Dean - CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr. M. Pushpalatha**, Professor and Associate Chairperson - CS, School of Computing and **Dr. Lakshmi,** Professor and Associate Chairperson - AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, **Dr. G Niranjana**, Professor & Department of Computing Technologies, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Project Coordinators, Panel Head, and Panel Members Department of Computing Technologies, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor, **Dr. S Ramesh**, Department of Computing Technologies, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr. M. Murali**, Department of Computing Technologies, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under her mentorship. She provided us with the freedom and support to explore the research topics of our interest. Her passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff members of Department of Computing Technologies, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21CSP302L – Project report titled **"SENTIMENTAL ANALYSIS USING SOCIAL MEDIA COMMENTS"** is the bonafide work of **"AMRITANSHU SHIWANSHI [RA2211003010756] and N SATYA DINESH [RA2211003010795]"** who carried out the project work under our supervision. Certifed further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation based on which degree or award was conferred on an earlier occasion on this or any other candidate.

Dr. M. Murali
Professor
Department of Computing Technologies
SRMIST

Dr. G Niranjana
Professor and Head
Department of Computing Technologies
SRMIST

Examiner I

Examiner II

TABLE OF CONTENTS

ABSTRACT

The research investigates the use of machine learning algorithms for the automation of sentiment classification from social media comments, which will help detect and analyze public opinion early enough to assist decision-making processes across different domains. The research focuses on an exhaustive comparative evaluation of three widely used machine learning architectures, namely Logistic Regression, Linear Support Vector Classifier (LinearSVC), and Random Forest, for the classification of three major sentiment classes: positive, negative, and neutral. The dataset applied for this work was obtained from Reddit, featuring thousands of user comments, and it had been organized into a clean CSV format with proper labeling, thereby aiding reproducibility and minimizing annotation errors. In addition, the dataset was divided into training, validation, and testing subsets in such a way that a stratified approach maintains class balance during evaluation. Each model was implemented once with default parameters and once with hyperparameter tuning, thus enabling the impact of algorithmic choices and training strategies to be rigorously evaluated. Preprocessing techniques, including text normalization, stopword removal, and TF-IDF vectorization, were performed to improve generalizability and robustness. All models were then assessed by means of accuracy, precision, recall, and F1-score, and confusion matrices were obtained for detailed error analysis. The results indicated that default LinearSVC produced the overall best accuracy with the highest value of 94.12%, giving the lowest class-wise imbalance performance, whereas tuned versions, especially tuned Random Forest, failed to improve and sometimes resulted in overfitting due to unsuitable hyperparameter tuning schemes. Of note, distinguishing neutral comments constituted the most challenging task across models, requiring specific class balancing techniques or improved text feature engineering methods. The project also emphasizes that standardized CSV organization with clean labeling gives better performance and fewer errors than any loosely structured datasets. The deductions made herein elaborate the way to determine successful automated sentiment analysis systems being based on scrutiny of model selection, hyperparameter control, and standardized dataset handling. The way is now clear for more studies on robust language models, imbalance-aware training, and real-world deployment across domains like marketing, mental health, and policy-making, thus adding further to the strides already being made in sentiment analysis from social media data.

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

AI – Artificial Intelligence

NLP – Natural Language Processing

BERT – Bidirectional Encoder Representations from Transformers

RoBERTa – Robustly Optimized BERT Approach

TF-IDF – Term Frequency-Inverse Document Frequency

EDA – Exploratory Data Analysis

SVR – Support Vector Regression

SDG – Sustainable Development Goals

WHO – World Health Organization

SMOTE – Synthetic Minority Over-sampling Technique

NLTK – Natural Language Toolkit

ROC – Receiver Operating Characteristic

CPU – Central Processing Unit

GPU – Graphics Processing Unit

Adam – Adaptive Moment Estimation (Optimizer)

CHAPTER 1

INTRODUCTION

1.1 Introduction

Over the past few years, the rise of social media platforms like Reddit and Twitter has created vast sources of user-generated text, offering insights into public opinion, emotional well-being, and even mental health signals. However, the informal, diverse, and often ambiguous nature of these texts makes understanding and classifying sentiments a challenging task. The problem becomes even more complex when certain sentiment classes—particularly neutral or negative—are underrepresented, leading to bias in model predictions. This growing complexity has fueled the demand for intelligent, scalable, and accurate solutions for sentiment classification in real-world environments.

Recent advances in Artificial Intelligence (AI), especially deep learning, have revolutionized how we process and analyze text data. Transformer-based models like BERT and RoBERTa have shown exceptional performance in understanding context-rich sentences, far surpassing traditional models. These architectures are particularly effective in sentiment analysis tasks, capable of interpreting subtle linguistic nuances in online discourse. Their adoption has opened the door to building robust sentiment detection systems that could benefit applications ranging from content moderation and public opinion mining to mental health support systems.

This study presents a comparative framework for sentiment classification using both traditional and deep learning models. The models evaluated include Logistic Regression, Random Forest, Support Vector Regression (SVR), and advanced architectures like BERT and RoBERTa. The classification models are trained on a preprocessed Reddit dataset, where user comments are labeled as positive, negative, or neutral. In addition to evaluating standard performance metrics such as accuracy, precision, recall, and F1-score, this work also addresses the problem of class imbalance using the RandomOverSampler technique to ensure fair training distribution across all sentiment classes.

A key focus of this work is understanding how model performance is affected by architectural complexity and training strategies. Although transformer models are powerful, their performance is

sensitive to factors like input sequence length, attention mechanisms, and learning rates. Improper tuning or lack of balance in data can lead to biased predictions or overfitting. To ensure reliable outcomes, this study incorporates careful preprocessing steps including text normalization, punctuation removal, and TF-IDF vectorization (for traditional models), followed by data balancing and model evaluation through confusion matrices and macro-averaged metrics.

Through rigorous experimentation and analysis, this project aims to draw a comprehensive comparison of the strengths and limitations of both traditional and transformer-based sentiment classifiers. By exploring how different models handle real-world Reddit data, this study contributes to the development of effective and interpretable sentiment analysis tools that can be applied in diverse applications.

Ultimately, this research seeks to bridge the gap between modern NLP techniques and practical use cases, enabling smarter tools for online sentiment tracking and emotional understanding. The insights derived from this project can inform the design of future systems aimed at enhancing digital well-being, improving content moderation, and supporting mental health awareness through intelligent analysis of social media content.

1.2 Problem Statement

Sentimental misinterpretation and lack of early detection of emotional distress expressed in social media content, particularly on platforms like Reddit, have become growing challenges in the digital public health domain. According to the World Health Organization, mental health conditions now represent a significant burden globally, and early detection is key to effective intervention and care. However, in many parts of the world—especially in rural or under-resourced communities—access to mental health professionals and sentiment monitoring systems is limited or entirely absent. As a result, countless individuals remain undiagnosed or unheard, which can escalate into severe mental health crises that might have been preventable with timely analysis and support.

Traditional sentiment classification and mental health monitoring still rely heavily on manual moderation and analysis by experts or community moderators. This approach is not only labor-intensive but also susceptible to inconsistencies, fatigue, and subjectivity. Moreover, the rise in social

media activity and the volume of user-generated content place increasing pressure on current digital infrastructure. This creates an urgent need for automated, scalable, and intelligent systems that can assist in understanding online sentiment and detecting signs of psychological distress in a timely manner.

Recent advances in Artificial Intelligence (AI), particularly with the introduction of deep learning and transformer-based models like BERT and RoBERTa, offer tremendous potential for developing sentiment detection systems with a high degree of contextual understanding and accuracy. Despite this, several technical and practical limitations persist. One such challenge is the sensitivity of these models to hyper-parameters such as learning rate, batch size, and sequence length. Poor tuning can lead to overfitting, underfitting, or unstable performance—reducing the reliability of such models in real-world applications.

Another significant issue is class imbalance in sentiment datasets. On platforms like Reddit, certain sentiments—especially neutral or negative expressions are underrepresented, while others (often general or positive sentiment) are dominant. When trained on such imbalanced data, even sophisticated models tend to favor the majority class, leading to high overall accuracy but poor performance in detecting minority sentiment classes. This is particularly dangerous when those minority classes contain signs of mental health distress, such as anxiety, depression, or suicidal ideation.

This project addresses these two pressing challenges—hyper-parameter sensitivity and class imbalance—in the context of sentiment analysis using Reddit comments. We build and compare both traditional machine learning classifiers (e.g., Logistic Regression, Random Forest, SVR) and advanced transformer models (e.g., BERT, RoBERTa), evaluating them under both default and hyper-parameter-tuned conditions. Using a curated Reddit dataset labeled for positive, negative, and neutral sentiments, we apply comprehensive preprocessing steps and use techniques like RandomOverSampler to balance class distributions. Each model's performance is evaluated using accuracy, precision, recall, F1-score, and confusion matrices.

The primary goal of this project is to create a robust, interpretable, and generalizable sentiment classification framework that can be practically deployed in mental health analysis, social listening

tools, and moderation systems. Through a comparative study of model behavior across architectures and configurations, we aim to highlight the strengths and limitations of each approach and provide guidelines for building ethical and effective AI solutions. Ultimately, this work seeks to support early detection of emotional distress, improve online well-being, and promote equitable access to digital mental health insights for all communities.

1.3 Motivation

Visual interpretation of user sentiment in social media comments has become increasingly significant in recent years, particularly with the surge in platforms like Reddit, where individuals frequently express opinions, emotions, and even psychological states. According to recent studies, social media content can provide early signals of public sentiment shifts and mental health concerns. However, in many areas especially those lacking digital infrastructure or human moderation capacity– identifying and interpreting online sentiment is still a major challenge. This can lead to the spread of harmful content, undetected negative behavior, or missed opportunities for early intervention in mental well-being monitoring.

The traditional method of sentiment classification often involves manual labeling and review, which is not only time-consuming but also subject to human error and inconsistency due to subjective interpretations. With the ever-increasing volume of online discourse, there is a growing demand for automated, efficient, and scalable tools that can support real-time sentiment tracking and analysis– tools that are both accurate and interpretable.

Recent developments in Artificial Intelligence (AI), especially through transformer-based models like BERT and RoBERTa, have significantly improved the ability to analyze text data with contextual understanding. These models have outperformed traditional machine learning algorithms in various natural language processing (NLP) tasks, including sentiment classification. However, the performance of such models is often sensitive to hyper-parameters like learning rate, batch size, and sequence length. Improper tuning can result in underperformance, overfitting, or instability, making deployment in live systems difficult without careful optimization.

Another persistent challenge is the issue of class imbalance in sentiment datasets. In platforms like Reddit, certain sentiment categories−such as neutral or negative−may be underrepresented compared to more common expressions. When trained on imbalanced datasets, even powerful models may become biased toward the majority class, resulting in high overall accuracy but poor performance on minority sentiments. This can be especially concerning when such sentiments are tied to critical issues like depression or public outrage.

This project addresses these two major challenges−hyper-parameter sensitivity and class imbalance−within the context of sentiment classification. We present a comparative analysis of both traditional classifiers (Logistic Regression, Random Forest, SVR) and transformer-based models (BERT, RoBERTa), evaluating them with and without hyper-parameter tuning. Using a labeled Reddit dataset, we preprocess and vectorize the data using TF-IDF for traditional models and tokenizer-based encoding for transformers. To address class imbalance, we employ RandomOverSampler techniques.

The models are assessed using accuracy, precision, recall, F1-score, and confusion matrices to provide a holistic view of their performance. By evaluating how each model performs under different configurations and data distributions, we aim to uncover best practices for building robust sentiment classification systems.

In summary, this project focuses on developing a reliable, generalizable, and interpretable sentiment analysis pipeline suitable for Reddit comments. By comparing model behavior under various tuning strategies and addressing class imbalance, we aim to contribute toward scalable AI systems that enhance content moderation, track public sentiment, and potentially assist in digital mental health interventions. Such tools could ultimately bridge the gap between real-time social media data and informed decision-making across domains like public policy, mental health support, and digital communication.

1.4 Sustainable Development Goals of the Project

This project directly supports the United Nations Sustainable Development Goal 3: Good Health and Well-being, which seeks to "ensure healthy lives and promote well-being for all at all ages." By enabling early detection of emotional distress and negative sentiment trends from Reddit comments, the project contributes to the broader goal of mental health awareness and intervention. Mental health conditions, particularly depression and anxiety, often manifest subtly in online discourse long before they are clinically diagnosed. Through AI-driven sentiment analysis, this project proposes a scalable system that can support early recognition of such signals—especially valuable in populations where professional psychological help is inaccessible or stigmatized.

By offering a reliable and automated platform for sentiment classification, the system assists in identifying emotional cues that may point to at-risk individuals or communities. This is crucial in underserved regions and online communities where access to mental health services is limited. Such a tool can empower moderators, support groups, or public health researchers to intervene earlier and more effectively, supporting the aim of universal mental health coverage under SDG Target 3.4, which focuses on reducing the burden of non-communicable diseases and promoting mental well-being.

The project also advances the principles of accessibility and equity in digital health. In many parts of the world, a significant divide exists in access to emotional and psychological support services. By reducing reliance on manual moderation and offering a system that can be integrated into various online platforms, this AI-driven approach helps bridge gaps in digital mental health care. It ensures that vulnerable and marginalized populations—especially those active in anonymous online spaces like Reddit—are not overlooked.

Furthermore, the project is committed to ethical AI practices. It prioritizes user privacy by working with publicly available and anonymized Reddit data, and addresses fairness by implementing class balancing techniques to avoid bias toward dominant sentiment categories. The use of explainable AI models enhances interpretability, ensuring that the system’s outcomes are transparent and trustworthy. These principles are in line with SDG 10: Reduced Inequalities, ensuring that technological benefits reach all groups fairly.

From a research and development standpoint, this work aligns with SDG 9: Industry, Innovation, and Infrastructure, demonstrating how advanced transformer models like BERT and RoBERTa can be applied to real-world sentiment classification problems. By fostering the application of AI in socially relevant domains, the project also supports SDG 4: Quality Education, promoting digital literacy and the responsible use of machine learning to tackle societal challenges.

In summary, this project showcases how artificial intelligence can be used not only for technological advancement but for social impact. By designing a sentiment analysis system that is inclusive, ethical, and scalable, it contributes meaningfully to global goals aimed at health, equity, and innovation−helping build a world where mental well-being and digital dignity are accessible rights for everyone.

CHAPTER 2
LITERATURE SURVEY

2.1 Overview of the Research Area

The research domain of this project falls at the intersection of Artificial Intelligence (AI), Natural Language Processing (NLP), and mental health informatics, aiming to improve the detection and classification of sentiment and potential emotional distress in online communities. Platforms like Reddit have become important spaces where people share their thoughts, experiences, and psychological states. This unstructured, real-time textual content often contains early indicators of emotional conditions such as depression, anxiety, or distress—many of which go unnoticed without scalable analysis tools. In digital environments lacking dedicated moderation or mental health monitoring, the need for intelligent, accessible, and efficient sentiment analysis systems is more pressing than ever.

Advancements in deep learning particularly transformer-based architectures and modern NLP pipelines—have revolutionized the ability to analyze unstructured text. Models like BERT and RoBERTa have demonstrated state-of-the-art performance in sentiment classification tasks due to their ability to capture deep contextual understanding. These models eliminate the need for handcrafted features and enable automated systems to detect sentiment and mood variations across large volumes of social media data. The application of such models to Reddit sentiment classification holds promise for enhancing content moderation, supporting mental health tracking, and enabling proactive digital well-being initiatives.

In this project, a comparison is conducted across traditional machine learning models (e.g., Logistic Regression, Random Forest, Support Vector Regression) and transformer-based architectures (BERT, RoBERTa) for automatic sentiment classification of Reddit comments. These models differ in their underlying complexity, interpretability, and computational requirements. The research adopts a comparative framework, where each model is evaluated in its default and hyperparameter-tuned configurations. The objective is to assess how these approaches perform under varied conditions and identify the most effective solution for accurate sentiment detection.

A core focus of this research is the investigation of hyperparameter sensitivity. Deep learning and machine learning models rely heavily on proper tuning of key parameters such as learning rate, regularization strength, batch size, and tokenization techniques. Improper tuning can lead to overfitting, underfitting, or poor generalization−hindering the reliability of the model. This study evaluates how different tuning strategies impact model behavior and how resilient each architecture is to changes in training conditions. Performance is assessed using metrics such as accuracy, precision, recall, F1-score, and validation loss.

Another critical issue addressed in this work is class imbalance−a common challenge in sentiment analysis datasets. In Reddit comment data, negative or neutral sentiments are often underrepresented, while positive sentiments may dominate due to sampling bias or data collection artifacts. Training on such imbalanced datasets leads to biased predictions where minority sentiments are frequently overlooked. To mitigate this, the project incorporates strategies like RandomOverSampler, stratified splitting, and balanced class weighting to enhance fairness and model generalization. Confusion matrices are generated to evaluate class-wise accuracy and ensure that models are not disproportionately favoring dominant sentiment categories.

The system developed through this research has real-world relevance in content moderation platforms, sentiment monitoring dashboards, and digital mental health support tools. In large-scale or underserved digital communities, where real-time intervention is critical, such an AI-based sentiment classification system can assist in flagging emotionally sensitive content, helping moderators respond more effectively and compassionately.

Beyond model performance, this project provides insights into the broader dynamics of NLP-based sentiment classification−highlighting how architecture selection, tuning methodology, and data balance directly affect the practical deployment of AI systems. The findings contribute to the design of inclusive, ethical, and scalable sentiment detection tools that can be integrated into existing digital infrastructure. These systems hold the potential to promote mental wellness, support early intervention strategies, and make emotional understanding tools accessible across diverse online communities.

2.2 Existing Models and Frameworks

The application of Artificial Intelligence (AI) in the analysis of human sentiment—particularly through Natural Language Processing (NLP)—has seen remarkable growth over recent years. Numerous models and frameworks have been proposed and deployed to automate sentiment classification from social media platforms such as Reddit. These frameworks increasingly rely on advanced deep learning methods, especially transformer-based models like BERT and RoBERTa, due to their exceptional capacity to understand linguistic structure, semantic context, and user intent from large-scale textual datasets. This project builds upon this growing body of work by focusing on three widely used model architectures: TF-IDF with Logistic Regression (custom-built), BERT, and RoBERTa. Each model brings a distinct architecture and has been extensively utilized in text classification tasks.

TF-IDF with Logistic Regression serves as the baseline model in this study. It utilizes term frequency-inverse document frequency vectors to represent text data and applies a linear classifier for sentiment prediction. Designed as a lightweight and interpretable architecture, this model is particularly suited for environments with limited computational resources or when quick model prototyping is required. While it lacks the depth of transformer models, TF-IDF models remain highly competitive for small to medium-scale sentiment tasks due to their simplicity and transparency.

BERT (Bidirectional Encoder Representations from Transformers), introduced by Google, revolutionized NLP by enabling bidirectional context-aware representations. BERT’s architecture processes input text by understanding both the left and right context simultaneously, making it well-suited for nuanced sentiment classification in Reddit comments. This model is evaluated in both its default pre-trained state and after fine-tuning to understand how transformer depth and context modeling improve classification accuracy.

RoBERTa, developed by Facebook AI, is an enhanced variant of BERT trained on larger corpora with optimized training dynamics. It discards the next-sentence prediction objective and increases training data volume, yielding significant improvements across sentiment benchmarks. RoBERTa is included in this research to assess how further architectural tuning and extensive training

improve performance in unstructured, socially diverse data environments like Reddit.

Beyond architecture, well-established libraries and frameworks are employed to implement and test these models. Transformers by Hugging Face, TensorFlow, and Scikit-learn provide robust support for training, hyperparameter tuning, tokenization, and evaluation. Visualization tools such as Matplotlib and Seaborn are used to create performance graphs, confusion matrices, and classification reports to provide detailed analysis of each models behavior.

The dataset utilized in this study consists of Reddit comments labeled for sentiment (positive, negative, neutral), offering a rich, real-world context for testing classification models. Reddit presents unique challenges due to its informal language, use of sarcasm, and unbalanced distribution of sentiment categories. To address class imbalance, techniques such as RandomOverSampler and class weighting are used to enhance fairness and boost recall for underrepresented sentiment classes.

By integrating these state-of-the-art models with rigorous experimentation, this project provides comparative insights into how various architectures behave under different configurations. The findings contribute to the optimization and advancement of NLP pipelines for sentiment analysis, and offer a foundation for deploying accurate, interpretable, and scalable AI solutions in social listening tools, mental health monitoring, and content moderation systems.

2.3 Research gaps identified

The literature surrounding sentiment analysis, especially in the context of social media platforms like Reddit, reveals several important research gaps that this project seeks to address. A prominent limitation across many existing studies is the insufficient handling of class imbalance. In sentiment datasets, especially those derived from Reddit, positive sentiments often dominate, while neutral or negative sentiments−which are crucial for understanding emotional distress or controversial discussions−are underrepresented. Many prior models tend to perform well on majority classes but exhibit poor recall for minority classes. Although techniques like SMOTE or RandomOverSampler exist, their usage and evaluation are often either superficial or omitted entirely, leading to skewed results that lack fairness and generalization.

Another notable gap is the lack of sensitivity analysis for hyperparameter tuning, particularly in the context of modern transformer models such as BERT and RoBERTa. While these architectures have set new benchmarks in natural language processing, many studies deploy them with default settings or minimal fine-tuning. As a result, there is limited understanding of how changes in learning rate, batch size, or token length impact performance—especially on domain-specific datasets like Reddit comments that differ significantly from cleaner benchmark corpora. This oversight results in under-optimized models that may not reflect their full potential.

Furthermore, many published works focus on structured and grammatically consistent datasets such as IMDb, Yelp, or Twitter, which do not reflect the chaotic, informal, and context-rich environment of Reddit. Reddit comments often include sarcasm, slang, emojis, and multi-threaded context, making them harder to analyze using traditional pipelines. However, very few studies in the literature take on the challenge of adapting models specifically to these intricacies. This gap highlights the need for more domain-aware preprocessing, robust vectorization techniques, and context-sensitive architectures.

In addition, existing research often lacks a clear comparative analysis between traditional machine learning models and modern transformer-based architectures. While deep learning dominates current literature, baseline comparisons with models like Logistic Regression, Support Vector Machines, or Naive Bayes—especially when paired with TF-IDF—are either absent or limited. This makes it difficult to understand the trade-offs between accuracy, interpretability, and resource consumption, especially when developing solutions for low-resource or real-time applications.

Lastly, there is a limited focus on ethical considerations and model interpretability in current sentiment analysis literature. In applications that intersect with mental health, misinformation detection, or emotional risk prediction, transparency and fairness are paramount. Yet, most studies emphasize raw performance metrics while overlooking how these models make decisions or how they might affect vulnerable users. Few incorporate explainable AI techniques or bias audits, leaving a crucial gap in building responsible and trustworthy sentiment analysis systems.

These research gaps form the foundation of this project’s objectives and experimental design. By addressing class imbalance, evaluating the impact of hyperparameter tuning, benchmarking across traditional and deep learning models, and emphasizing ethical AI, this study aims to contribute meaningfully to the evolving landscape of sentiment analysis in social media environments like Reddit.

2.4 Research Objectives

The overall objective of this research project is to develop, deploy, and evaluate an AI-based sentiment classification system capable of automatically detecting and categorizing emotional tones—such as positive, negative, and neutral—from Reddit comments using both traditional and deep learning methods. Specifically, the project aims to implement and compare diverse classification models, including TF-IDF with Logistic Regression, BERT, and RoBERTa, to assess their performance in understanding and interpreting user sentiment within real-world, informal social media text. This study addresses two major technical challenges: hyperparameter sensitivity and class imbalance, both of which are often underexplored in existing sentiment analysis literature.

One of the primary objectives is to compare the predictive performance of each model using standardized evaluation metrics such as accuracy, precision, recall, and F1-score. Through a comprehensive analysis of classification results across the sentiment classes, the research aims to determine which architecture delivers the most accurate, stable, and computationally efficient performance. In addition to default configurations, this work examines the effects of hyperparameter tuning—such as adjustments to learning rate, batch size, dropout rate, and number of training epochs—on training behavior and final prediction quality.

A central focus of the study is to explore model sensitivity to hyperparameter variation. While many studies report high accuracy using pre-trained language models, few investigate how performance fluctuates when tuning is poorly optimized or highly refined. This project bridges that gap by conducting controlled experiments under both tuned and untuned conditions—particularly for complex models like BERT and RoBERTa—offering insights into the optimization strategies needed to maintain performance consistency and generalizability across different datasets or domains.

Another critical objective is to address the class imbalance commonly present in social media sentiment datasets. In Reddit data, positive sentiment often dominates, while negative and neutral classes may be underrepresented. This imbalance can bias models toward the majority class, resulting in poor detection of minority sentiments, which are particularly relevant in identifying distress or harmful behavior. Techniques such as RandomOverSampler, class weighting, and balanced data splitting are used in this study to reduce this bias, and model performance is further analyzed through confusion matrices and per-class evaluation reports to ensure balanced outcomes.

The project also aims to build an end-to-end modular training and evaluation pipeline that includes text preprocessing, feature extraction, model training, and results visualization. Preprocessing involves standard techniques such as lowercasing, punctuation removal, tokenization, and vectorization. The pipeline is designed to be scalable and adaptable, allowing for the integration of additional models or datasets in future iterations. Visualization tools such as classification reports and heatmaps are included to support both qualitative and quantitative performance assessment.

Beyond technical development, this project has practical implications for fields like mental health analysis, social media monitoring, and content moderation. By evaluating the trade-offs between simpler models (e.g., TF-IDF + Logistic Regression) and deeper models (e.g., BERT, RoBERTa), the study contributes insights into which architectures are most feasible for deployment in low-resource environments or real-time systems. The evaluation includes not only predictive performance but also considerations of interpretability, training time, and scalability−ensuring that the developed solutions are not just accurate, but practically deployable.

In essence, this research seeks to bridge the gap between academic sentiment modeling and real-world social media applications. By addressing both technical and operational challenges, it provides a foundation for building robust, ethical, and scalable sentiment analysis tools that can assist moderators, mental health professionals, and AI practitioners in making informed, timely, and empathetic decisions based on user-generated text.

2.5 Product Backlog (Key User Stories with Desired Outcomes)

In the development of our AI-powered sentiment analysis system for Reddit data, the product backlog outlines a structured roadmap of prioritized functionalities and research tasks. Each item is framed as a user story that addresses a specific need or technical goal, along with the desired outcomes. These user stories guide the project toward delivering a research-driven, scalable, and impactful NLP solution for classifying sentiment in social media content using both traditional and transformer-based models.

User Story 1:

We as researchers would like to preprocess and curate a clean, standardized, and sentiment-labeled Reddit dataset so that models are trained on consistent and representative input.

Desired Outcome: A fully preprocessed dataset (cleaned, tokenized, normalized, and balanced), split into training, validation, and test sets, ensuring minimal noise and representative sentiment distribution across classes.

User Story 2:

We as data scientists want to train and compare multiple models (TF-IDF + Logistic Regression, BERT, RoBERTa) to evaluate their effectiveness on Reddit sentiment classification.

Desired Outcome: All models are trained and evaluated on the same dataset, with performance metrics (accuracy, precision, recall, F1-score, confusion matrix) documented to identify the most robust architecture.

User Story 3:

We as model developers aim to perform hyperparameter tuning (e.g., learning rate, batch size, dropout rate) to identify the optimal training settings for each model.

Desired Outcome: Systematic tuning trials are conducted, showing measurable performance gains or losses, with tuning insights recorded for future replication or fine-tuning.

User Story 4:

We as researchers want to address class imbalance in sentiment data so that underrepresented sentiments (e.g., negative, neutral) are fairly classified.

Desired Outcome: Techniques like RandomOverSampler or class weighting are applied, resulting in improved per-class performance, especially for minority sentiments, as shown in detailed classification reports.

User Story 5:

We as evaluators want to create visual and statistical reports for each model to aid in communication with both technical and non-technical stakeholders.

Desired Outcome: Confusion matrices, bar charts, ROC curves, and classification summaries are generated and formatted clearly to support analysis and presentation.

User Story 6:

We as planners want to assess the computational efficiency of each model to ensure practical deployment, particularly in low-resource or real-time environments.

Desired Outcome: Benchmark results on training/inference time, memory usage, and model size are captured to inform deployment strategies.

User Story 7:

We as ethical AI practitioners want to ensure our system upholds data privacy and fairness to align with responsible AI standards.

Desired Outcome: Data use is transparently documented, Reddit data is anonymized and sourced ethically, and bias mitigation strategies are embedded in both design and reporting.

User Story 8:

We as technical writers want to capture the methodology, model design, and findings in a comprehensive research report.

Desired Outcome: A complete document is compiled, including the problem definition, literature survey, experimental results, architecture diagrams, and conclusions.

User Story 9:

We as software engineers want to build a modular evaluation framework combining all models for easy comparison and experimentation.

Desired Outcome: A unified Python codebase with components for preprocessing, training, evaluation, and visualization is developed and published for reuse.

User Story 10:

We as AI researchers want to perform error analysis to investigate why and where models misclassify certain sentiments.

Desired Outcome: A detailed error breakdown is created, with insights into misclassified cases, especially for neutral and negative classes, guiding future refinement.

User Story 11:

We as communicators want to prepare visual-rich presentation slides summarizing the research approach and outcomes.

Desired Outcome: A polished presentation with charts, architecture flows, and bullet-point highlights is delivered for academic or public dissemination.

User Story 12:

We as contributors want to summarize final findings and draw meaningful conclusions from the research.

Desired Outcome: The project is wrapped up with clearly stated learnings on model comparison, performance trade-offs, limitations, and suggested future work.

User Story 13:

We as the project team want to finalize and submit all deliverables in a well-documented and organized manner.

Desired Outcome: All files−including source code, performance reports, documentation, presentations, and final research papers−are bundled, reviewed, and submitted on time.

2.6 Plan of Action (Project Roadmap)

Effective project execution was carried out through a methodical and time-sensitive strategy, allowing structured progression through every stage−from dataset acquisition and preprocessing to model development, evaluation, documentation, and deployment readiness. The project followed a weekly sprint plan spread across eight weeks, with each sprint focusing on a set of well-defined tasks aligned with our overarching goal of building an AI-based sentiment classification system using Reddit comments.

Week 1: Project Initialization and Dataset Exploration

The project began with defining the problem and clarifying our research objectives. We conducted a literature review on sentiment classification models, particularly on Reddit-based NLP research and transformer architectures like BERT and RoBERTa. A suitable sentiment-labeled Reddit dataset was selected for experimentation. We explored its structure, label distribution, and token-level characteristics, noting issues such as class imbalance and informal language patterns common in Reddit comments.

Week 2: Dataset Preprocessing and Splitting

We cleaned the dataset by performing text normalization steps, including lowercasing, punctuation removal, and tokenization. For traditional models, TF-IDF vectorization was applied, while for transformer-based models, token encoding was handled via pre-trained tokenizers. The dataset was split into training, validation, and test sets (80-10-10), and sentiment distribution across these subsets was visualized to ensure consistency

Week 3: Baseline Model Development

Three baseline models−TF-IDF with Logistic Regression, BERT, and RoBERTa−were implemented using their default configurations. Each model was trained on the same data for performance benchmarking. We tracked training accuracy and loss, and evaluated model predictions using classification reports, confusion matrices, and F1-scores to establish a baseline for further tuning.

Week 4: Hyperparameter Tuning and Retraining

In this sprint, we focused on optimizing hyperparameters such as learning rate, batch size, sequence length, and epochs. Experiments were run to assess how each tuning choice affected model convergence and accuracy. The tuned versions of all models were retrained and re-evaluated. Improvements in validation accuracy and per-class recall were tracked to understand each model’s sensitivity to tuning.

Week 5: Addressing Class Imbalance and Advanced Metrics

We applied RandomOverSampler, class weighting, and stratified sampling to mitigate class imbalance in the training data. After retraining, we evaluated how these strategies improved model fairness−especially in predicting underrepresented sentiment classes like "negative" or "neutral." Confusion matrices, macro-averaged F1-scores, and per-class analysis were generated, highlighting misclassification trends and bias reduction.

Week 6: Error Analysis and Comparative Evaluation

This week was dedicated to detailed error analysis. We compared baseline vs. tuned models in terms of precision, recall, and F1-score. Special attention was given to misclassified samples, especially cases of false negatives in negative sentiment detection. Based on performance metrics and computational efficiency, the most balanced model for deployment was identified.

Week 7: Documentation and Ethical Considerations

All experimental findings, methodologies, model architectures, evaluation plots, and project insights were compiled into a final report. We also addressed ethical AI principles, documenting privacy measures (use of anonymized public Reddit data), fairness considerations, and potential real-world

impacts. The project was also mapped to Sustainable Development Goal 3: Good Health and Well-being, particularly in the context of digital mental health support.

Week 8: Finalization and Submission

In the final sprint, we created a well-structured project presentation summarizing our problem statement, models, results, and recommendations. Code, model checkpoints, performance visualizations, and written documents were consolidated, reviewed, and finalized. A final retrospective was conducted to reflect on the journey, highlight lessons learned, and evaluate future research directions.

This structured sprint plan (Fig. 1) enabled us to manage the project efficiently, meet research milestones, and ensure timely completion of all deliverables while addressing both technical and ethical goals central to sentiment analysis in social media contexts.

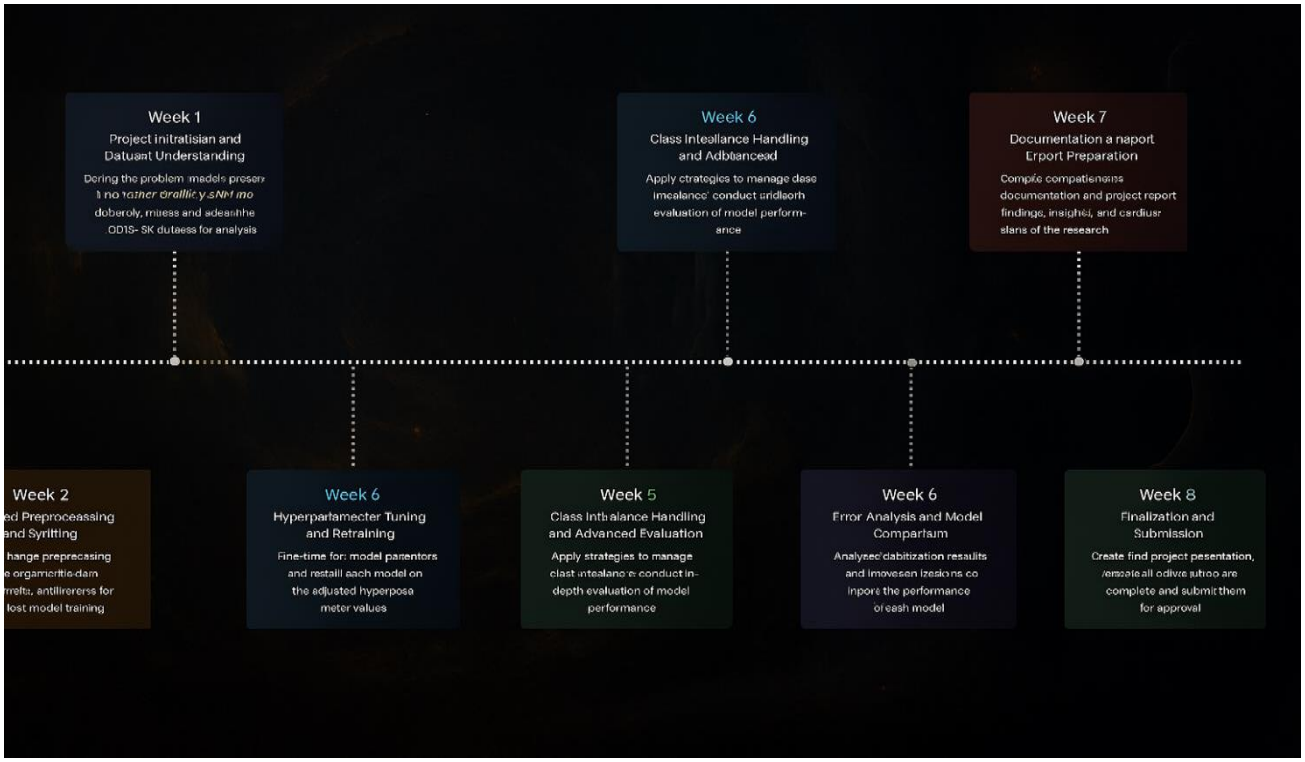


Fig 1. Project Roadmap

CHAPTER 3

SPRINT PLANNING EXECUTION AND METHODOLOGY

3.1 SPRINT I

3.1.1 Sprint I Objectives with User Stories

The first sprint laid the groundwork for all subsequent stages of model development and experimentation. The core objective in Sprint I was to establish a strong foundation for building an AI-driven sentiment classification system for Reddit comments using both traditional and transformer-based models. This phase involved dataset acquisition, exploratory analysis, establishing preprocessing pipelines, and defining performance metrics tailored to sentiment analysis tasks.

Our first key task was to obtain and preprocess the Reddit sentiment dataset. We selected a publicly available, labeled dataset comprising Reddit comments categorized into positive, negative, and neutral sentiment classes. The dataset was reviewed to ensure proper labeling, cleaned of irrelevant entries, and verified for structural consistency. We also organized the data in a directory and dataframe format suitable for feeding into both TF-IDF vectorizers and transformer tokenizers, ensuring smooth model integration in later phases.

A second major milestone of this sprint was to analyze the class distribution and linguistic traits of the dataset through exploratory data analysis (EDA). We examined comment lengths, sentiment frequencies, and vocabulary richness. The analysis revealed a noticeable class imbalance, where negative and neutral sentiments were significantly underrepresented compared to positive comments. We flagged this as a potential source of bias and documented it for mitigation in future sprints via oversampling and weighted loss functions.

The third focus was to implement a robust text preprocessing pipeline. We applied standard NLP cleaning steps including lowercasing, punctuation removal, stopword elimination, and tokenization. For traditional models like Logistic Regression, we vectorized the text using TF-IDF,

while for BERT and RoBERTa, we used their respective pre-trained tokenizers to handle encoding with attention masks and padding. Special care was taken to avoid data leakage, ensuring preprocessing was only applied to training data during model training.

Finally, we defined the evaluation criteria and model objectives for the entire project. The goal was to classify any given Reddit comment into one of the three sentiment categories and assess model performance using Accuracy, Precision, Recall, and F1-score. Emphasis was placed on Recall and F1-score, particularly for underrepresented sentiment classes, as these metrics are essential when the goal is to detect subtle or rare emotional signals—such as expressions of distress, frustration, or mental health concerns—in online discourse.

This sprint successfully set up the tools, structure, and direction required for deep experimentation in subsequent sprints, enabling us to proceed with confidence in tuning, training, and evaluation phases.

3.1.2 Functional Document

One of the most critical deliverables developed in Sprint I was the data ingestion and preprocessing pipeline for Reddit sentiment analysis. This pipeline served as the foundation for transforming raw Reddit comment data into a clean, tokenized, and vectorized format, compatible with both traditional machine learning models and deep learning transformers. Given our project’s goal of deploying and comparing TF-IDF with Logistic Regression, BERT, and RoBERTa, it was essential to ensure that the pipeline was modular, extensible, and robust across different model configurations.

The pipeline began by loading and inspecting the raw dataset, which consisted of Reddit comments labeled with sentiments: positive, negative, and neutral. We performed thorough data verification to handle missing labels, remove null entries, and ensure consistent formatting of textual inputs. All data was organized into a structured DataFrame format, making it compatible with vectorization and tokenization libraries such as Scikit-learn’s TfidfVectorizer and Hugging Face Transformers' Tokenizer APIs.

Following ingestion, the preprocessing phase included multiple stages of text normalization. This involved converting text to lowercase, removing punctuation and special characters, stripping unnecessary whitespace, and filtering out stopwords using NLTK. For traditional models, we used TF-IDF vectorization to convert cleaned text into numerical features. For deep learning models like BERT and RoBERTa, we employed pre-trained tokenizers, which handled text truncation, padding, and creation of attention masks in accordance with the models' input requirements.

The data was then split into training (80%), validation (10%), and test (10%) sets, ensuring class balance across each subset. This stratified split preserved the natural distribution of sentiment labels, enabling a fair performance comparison and preventing models from overfitting to majority classes. Care was taken to avoid data leakage by isolating preprocessing logic for the training set from that used during evaluation and testing.

To tackle class imbalance, particularly the underrepresentation of negative and neutral sentiments, we implemented techniques like RandomOverSampler and class weighting within training procedures. These adjustments helped ensure better recall and F1-scores for minority classes, which are crucial for accurate sentiment interpretation and mental health monitoring.

The pipeline was further tested under various batch sizes−16, 32, and 64−to evaluate its compatibility with different hardware environments and ensure stability across both CPU and GPU setups. Efficient memory usage, especially during tokenization and model input preparation, was validated to prevent runtime errors such as sequence truncation issues or memory overflows during batching.

We also developed a suite of custom utility functions for visualization and debugging. These included functions to display pre-tokenized and post-tokenized samples, highlight token padding effects, and view batch distributions by sentiment class. These visual tools proved invaluable in confirming preprocessing effectiveness and aided in early detection of data-related anomalies.

Every component of the preprocessing pipeline was written to be modular and reusable, with each function handling an isolated task−text cleaning, tokenization, vectorization, batching, or class

balancing. Global parameters, such as maximum sequence length, batch size, and preprocessing rules, were maintained in a centralized configuration file to ensure consistency and reproducibility across experiments and deployments.

This modular pipeline formed the backbone of our entire experimental workflow. It ensured reliable, clean, and scalable input preparation across all sentiment classification models used throughout the project. With the preprocessing system fully implemented and validated by the end of Sprint I, we were ready to advance toward model training, tuning, and evaluation in the following phases.

3.1.3 Architecture Document

The architecture developed during Sprint I laid the foundational framework for the entire Reddit sentiment analysis system. The primary objective of the architecture was to build a modular, flexible, and extensible framework that could support a variety of machine learning workflows involving both traditional classifiers (like TF-IDF + Logistic Regression) and deep learning transformers (like BERT and RoBERTa). Given the research-focused nature of this project, which includes model benchmarking, hyperparameter tuning, and error analysis, it was crucial to construct a system that allowed rapid iteration and scalability, while maintaining clarity and reusability across different components.

To ensure clean development and easier collaboration, the entire system was broken down into independent modules, each responsible for a specific stage of the sentiment analysis pipeline. This design facilitated debugging, experiment tracking, and team-based parallel development. It also allowed each component to be tested or replaced without affecting the entire pipeline.

At the center of the system was the Data Ingestion Module. This component was responsible for loading the Reddit dataset, verifying data integrity, and structuring the input text and associated sentiment labels (positive, negative, neutral). The data was validated for missing entries, duplicate records, and malformed text. Once verified, the data was organized in a clean format compatible with both Scikit-learn’s TfidfVectorizer and Hugging Face Tokenizers, ensuring flexibility across model types.

The Preprocessing Module was another core component. It handled text normalization tasks including lowercasing, punctuation removal, stopword filtering, and tokenization. For traditional models, the text was vectorized using TF-IDF, while transformer models received tokenized input using BERT and RoBERTa tokenizers, which generated attention masks and padded sequences as required. These steps were configurable based on whether the pipeline was used for training, validation, or testing, ensuring no leakage across data splits.

We also implemented an Exploratory Data Analysis (EDA) Module, which played a key role in understanding the dataset before training began. This module included utilities for visualizing sentiment class distributions, comment length histograms, and token frequency plots. Random samples were also printed and reviewed to check for content quality, labeling consistency, and outliers such as offensive or off-topic text. This visual and statistical feedback was critical in identifying class imbalance and planning mitigation strategies like oversampling and reweighting.

To maintain consistency across experiments, we used a centralized Configuration Script to house all critical settings. This included values such as maximum sequence length, batch size, learning rate, number of sentiment classes, directory paths, and model-specific flags (e.g., tokenizer type or dropout rate). This approach minimized errors during experimentation and made it easy to reproduce results or share setups across collaborators.

The system was built using a robust Python-based technology stack. We used Scikit-learn and Transformers (Hugging Face) for model implementations, NumPy and Pandas for data handling, Matplotlib and Seaborn for visualization, and NLTK for basic text preprocessing. This allowed us to process large volumes of comment data efficiently, transform raw inputs in real time, and generate detailed analytical outputs.

To support collaborative development, the project was managed using Git and hosted on a shared GitHub repository. Team members worked on individual branches, contributing features, tuning experiments, or fixing bugs in isolation, which were then merged into the main branch after code

review. This strategy supported effective version control, parallel development, and safe experimentation.

By the end of Sprint I, the architectural design had successfully delivered a clean, modular, and scalable sentiment analysis environment. It enabled structured experimentation with various models and positioned the team for seamless advancement into model training, performance evaluation, and hyperparameter tuning in Sprint II.

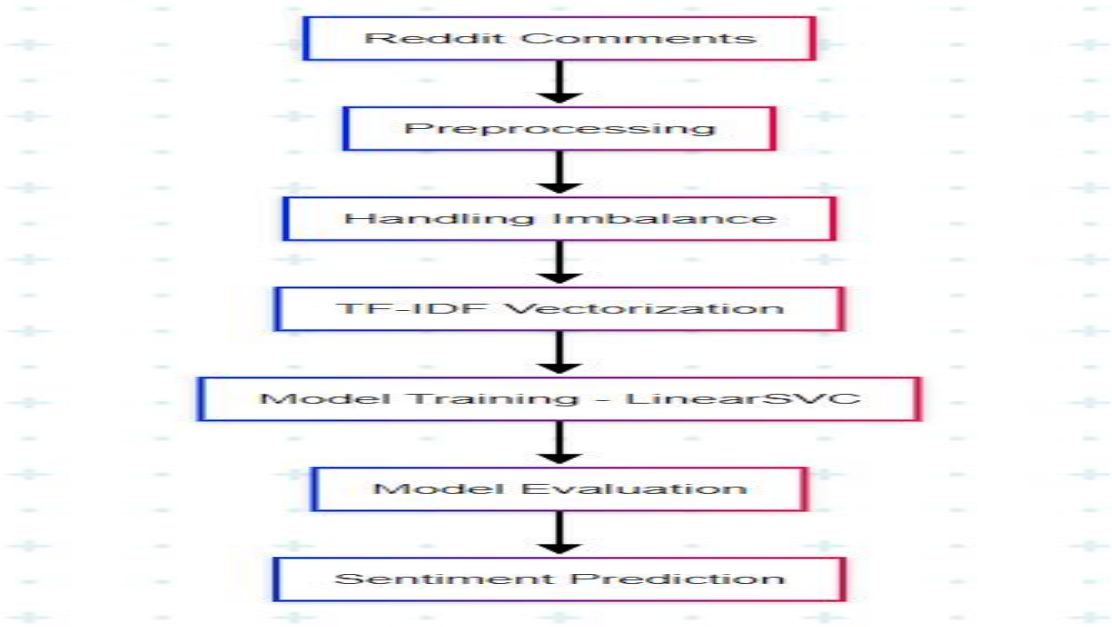


Fig. 3.1.3 Architecture Diagram

3.1.4 Result Analysis

By the conclusion of Sprint I, we had successfully developed a robust, reusable, and modular text preprocessing and data handling pipeline, which laid the groundwork for all future model training and evaluation activities. This pipeline incorporated clearly defined stages including data loading, cleaning, tokenization, and vectorization, all of which were independently tested and validated. It was built to support both traditional models like TF-IDF with Logistic Regression and advanced transformer-based models such as BERT and RoBERTa, ensuring flexibility for different experiment types.

The preprocessing pipeline was validated using visualization tools, such as token distribution plots, sentence length histograms, and TF-IDF feature density views. These visual checks confirmed that the text cleaning steps−like punctuation removal, lowercasing, stopword elimination, and token truncation−were functioning as intended. Additionally, the BERT and RoBERTa tokenizers were tested to ensure correct encoding of Reddit comments, including special tokens, attention masks, and padding behaviors. This ensured that no meaningful linguistic features were lost or distorted during preprocessing.

Using exploratory data analysis (EDA), we identified a clear class imbalance, with positive comments being overrepresented compared to neutral and negative ones. This imbalance presented a major challenge for later stages, particularly given the real-world importance of detecting distress or harmful sentiments−often hidden in minority classes. Visualizations such as bar plots and pie charts made these imbalances evident, helping us plan effective oversampling and class-weighting strategies for Sprint II.

The dataset was successfully split into training (80%), validation (10%), and test (10%) sets using a stratified sampling strategy. This ensured that sentiment class distributions remained consistent across all subsets, enabling unbiased evaluation and reducing the risk of performance skew. This structure also allowed us to clearly separate training, tuning, and testing stages, supporting reproducibility and fairness across all model comparisons.

The modular design of the pipeline allowed team members to work on separate components in parallel−such as preprocessing, tokenizer testing, or TF-IDF experimentation−leading to faster development and easier collaboration. All key parameters, including batch size, max sequence length, and tokenizer type, were stored in a central configuration file, which helped maintain consistency across experimental runs and simplified future hyperparameter tuning.

We also finalized a set of evaluation metrics to be used in upcoming model assessments. These included Accuracy, Precision, Recall, and F1-score, with an emphasis on macro-averaged and class-specific values. Particular focus was given to Recall and F1-score for negative and neutral sentiment

classes, as misclassifying these could lead to overlooking critical emotional cues, especially in mental health contexts.

Overall, Sprint I successfully delivered a technically sound and operationally efficient foundation for future phases. The data pipeline, EDA tools, and evaluation strategy established in this sprint aligned perfectly with our project roadmap and enabled us to transition seamlessly into Sprint II, where model training, tuning, and comparison would take center stage.

3.1.5 Sprint Retrospective

Sprint I served as a highly productive and foundational phase of our project, establishing a well-organized and scalable infrastructure upon which the rest of our sentiment analysis research would be built. The primary focus of this sprint was to prepare the environment for upcoming model experimentation through effective data acquisition, cleaning, exploratory analysis, and preprocessing of Reddit comments. By the end of this phase, we had developed a **modular, reusable text preprocessing pipeline** that could be easily integrated with both traditional and transformer-based sentiment classifiers.

One of the most notable accomplishments of this sprint was the completion of a **robust data ingestion and preprocessing system**. This system handled the complexities of noisy and informal Reddit text by performing cleaning tasks such as punctuation removal, lowercasing, and stopword filtering. It also supported both TF-IDF vectorization for lightweight models and tokenizer-based encoding for advanced transformer architectures like BERT and RoBERTa. This flexibility ensured that the preprocessing pipeline could serve as a backbone for all upcoming model training workflows.

A further strength of the sprint was the incorporation of **exploratory data analysis (EDA)** tools that helped us understand the underlying sentiment class distribution, comment length variability, and token frequency. Visual plots generated through EDA revealed a significant **class imbalance**, where neutral and negative sentiments were notably underrepresented. These insights directly informed our planning for Sprint II, where mitigation techniques such as RandomOverSampler and class weighting would be implemented.

Although we met all core objectives for this sprint, there were some **implementation challenges**. Cleaning and validating text entries took more time than anticipated due to inconsistent formatting and occasional missing sentiment labels. Additionally, tuning tokenizer parameters—such as maximum sequence length and padding strategy—required several iterations to ensure alignment across training and evaluation batches. Despite these delays, the challenges led to meaningful learning and a better understanding of the intricacies of working with real-world social media data.

One important takeaway was the **importance of building modular and flexible components** from the outset. Since we anticipated using multiple models with different input requirements, we designed preprocessing and data loading modules to be easily adjustable. Whether switching from TF-IDF to BERT, or modifying batch sizes and sequence lengths, the reusable nature of these modules saved significant time and allowed for streamlined experimentation in later sprints.

Sprint I also clarified the technical specifications for model training, highlighted key risks (e.g., data imbalance and token truncation), and positioned the team to transition smoothly into **Sprint II**. The upcoming sprint would focus on implementing baseline models, conducting initial performance evaluations, and comparing their effectiveness using metrics such as Accuracy, Precision, Recall, and F1-score—especially on the minority sentiment classes.

In summary, Sprint I was a crucial milestone that aligned the project with its broader goal of creating a **scalable, interpretable, and socially relevant sentiment classification system** for Reddit. With preprocessing, data validation, and metric definitions now in place, we were fully prepared to begin model development and experimentation in the next sprint.

3.2 SPRINT 2

3.2.1 Sprint 2 Objectives with User Stories

Sprint II marked a pivotal phase in the development of our Reddit-based sentiment analysis project. Building on the preprocessing pipeline and exploratory data analysis completed in Sprint I, this sprint transitioned into the core machine learning stage−model implementation, training, and evaluation. The overarching goal was to integrate both traditional and deep learning architectures into the workflow, assess their performance under controlled conditions, and derive actionable insights about their suitability for sentiment classification on social media text.

User Story 1

As a data scientist, I want to build and train a lightweight traditional model using TF-IDF with Logistic Regression so that I can establish a baseline performance on the Reddit sentiment dataset.

Objective: Construct a TF-IDF vectorizer pipeline and train a logistic regression classifier to serve as a quick, interpretable benchmark.

User Story 2

As an NLP engineer, I want to fine-tune a pre-trained BERT model on the Reddit sentiment dataset so that I can leverage contextual embeddings for more accurate sentiment detection.

Objective: Integrate Hugging Face’s BERT tokenizer and classifier, train it using encoded Reddit data, and monitor performance using standard NLP metrics.

User Story 3

As a researcher, I want to train a RoBERTa model under the same experimental settings as BERT so that I can compare the transformer variants in terms of effectiveness and resource cost.

Objective: Run RoBERTa using identical input structure, training epochs, and batch sizes to ensure a fair comparison with other models.

User Story 4

As a model evaluator, I want to compare TF-IDF, BERT, and RoBERTa models across consistent metrics like accuracy, precision, recall, and F1-score so that I can evaluate model competence and fairness.

Objective: Compute macro-averaged and per-class evaluation scores across all models, focusing especially on underrepresented sentiment classes like negative and neutral.

User Story 5

As an analyst, I want to generate confusion matrices and performance plots so that I can visually interpret where each model performs well or poorly.

Objective: Use Seaborn and Matplotlib to plot confusion matrices, ROC curves, and bar graphs to support data-driven discussions of each model’s strengths and weaknesses.

User Story 6

As a research lead, I want to maintain equivalent training conditions (batch size, optimizer, learning rate) across all models to ensure experimental fairness.

Objective: Standardize hyperparameters like learning rate (2e-5), batch size (16), optimizer (Adam), and epoch count (3-5) across model runs.

User Story 7

As an AI practitioner, I want to implement early stopping and model checkpointing so that training halts automatically on performance degradation and best weights are preserved.

Objective: Add callbacks that track validation loss to avoid overfitting and ensure reproducibility of best-performing models.

User Story 8

As a reviewer, I want to identify how each model handles class imbalance so that I can understand the effectiveness of oversampling and class-weighting strategies.

Objective: Compare model outputs, especially recall and F1-score on minority sentiment classes, and analyze trade-offs between fairness and accuracy.

By the end of Sprint II, we had trained and tested all three models−TF-IDF + Logistic Regression, BERT, and RoBERTa−under consistent, controlled conditions. Evaluation was performed using Accuracy, Precision, Recall, and F1-score, with additional emphasis on class-wise performance analysis. Confusion matrices and visual outputs gave meaningful insight into classification behaviors and shortcomings of each architecture.

This sprint ultimately addressed the core research question: which models are most effective for sentiment classification on unstructured, imbalanced Reddit data, and what are the trade-offs between performance, interpretability, and resource efficiency. The results informed our decisions for presentation, documentation, and potential real-world deployment of the sentiment analysis system.

3.2.2 Functional Document

Sprint II focused on the **implementation and validation of core sentiment classification models**. This phase marked the transition from data preprocessing to actual model deployment for multi-class sentiment prediction. The aim was not only to build the models but also to maintain **experimental consistency** and enable **fair comparison** by using a uniform training and evaluation pipeline.

The first functional element implemented was a **TF-IDF with Logistic Regression pipeline**. This model served as a lightweight and interpretable benchmark. Using TfidfVectorizer from Scikit-learn, Reddit comments were transformed into sparse matrices of token importance, and a LogisticRegression classifier was trained to predict sentiment (positive, negative, neutral). Despite its simplicity, this model offered rapid training and interpretability, making it ideal for setting a performance baseline and understanding the feature space.

Next, two **transformer-based models** were introduced: **BERT (Bidirectional Encoder Representations from Transformers)** and **RoBERTa**. These were accessed via the Hugging Face

transformers library and fine-tuned on the Reddit dataset using the Trainer API. Each model utilized its pre-trained tokenizer to encode comments into token IDs, along with attention masks for padding. The classification heads were redefined to align with our 3-class output format.

To ensure **comparability across all three models**, we standardized the training process. For all models, we used a consistent **batch size of 16**, **learning rate of 2e-5**, and **training duration of 3–5 epochs**. The **Adam optimizer** was used for its adaptive learning capability, while **cross-entropy loss** was employed for handling categorical targets.

To improve stability and optimize performance, we implemented key training callbacks such as **EarlyStopping** and **ModelCheckpoint**. EarlyStopping monitored the validation loss and halted training once performance plateaued, preventing overfitting. ModelCheckpoint ensured that only the best-performing model (based on validation loss) was saved for final testing.

A centralized **evaluation module** was developed to ensure consistent metrics across all experiments. It calculated **accuracy, precision, recall, and F1-score** both overall and per class, enabling robust comparison between models. **Confusion matrices** were also generated for each model to visualize misclassifications and highlight sentiment classes that were consistently confused, especially between neutral and negative comments.

In addition, **visual monitoring tools** were developed to plot training and validation accuracy/loss over time. These real-time plots were invaluable for debugging and helped identify underfitting, overfitting, or issues with model convergence. They also supported decisions on early stopping and hyperparameter refinement.

The pipeline was constructed with a **modular and scalable architecture**. Each component—from text vectorization to model training and evaluation—was encapsulated in independent, reusable functions. This design allowed minimal modification when switching between models or hyperparameters, and supported rapid experimentation without restructuring code.

In summary, the functionality implemented in **Sprint II** empowered the team to run fair, reproducible, and scalable experiments across both traditional and deep learning models. With this flexible architecture in place, we were able to **focus more on analysis and interpretation**, reducing development friction and laying the foundation for final model selection and reporting.

3.2.3 Architecture Document

The architecture developed in Sprint II was significantly expanded to support the integration, experimentation, and evaluation of multiple sentiment classification models. While Sprint I laid the foundation with a modular preprocessing and EDA pipeline, Sprint II added model-specific layers to facilitate training and testing of both traditional and transformer-based NLP models. This enhanced system architecture was designed to ensure modularity, reproducibility, and scalability across experiments.

A central component introduced in this phase was the Model Integration Layer. This layer abstracted the implementation details of individual models—including TF-IDF with Logistic Regression, BERT, and RoBERTa—and provided a unified interface to invoke them within a common framework. Each model was encapsulated in its own module, allowing for quick swapping and minimal code changes when experimenting with different architectures.

To manage experiments efficiently, a Central Controller Script was created as the single entry point for initiating model training and testing. By modifying parameters like model type, batch size, and epoch count, developers could run end-to-end experiments with ease. This reduced manual overhead and ensured consistency across training sessions.

Evaluation tools were also formalized into a dedicated Evaluation Module. This module automated the computation of metrics such as accuracy, precision, recall, and F1-score, both globally and per sentiment class. It also generated visual outputs including training curves and confusion matrices, which were essential for interpreting model performance and identifying weak areas such as the consistent misclassification of neutral sentiments.

To improve training reliability and efficiency, a suite of callbacks was implemented. EarlyStopping was used to terminate training once validation loss plateaued, while ModelCheckpoint saved the best weights during training for final evaluation. For transformer models, learning rate schedulers like ReduceLROnPlateau helped optimize convergence by lowering the learning rate when progress stalled. These tools ensured efficient resource use and stable model performance.

All models were trained under standardized settings using the Adam optimizer and categorical cross-entropy loss to enable fair performance comparisons. Hyperparameters like batch size, learning rate, and number of epochs were kept uniform across all models to avoid introducing bias into the evaluation results.

The architecture continued to leverage the technology stack introduced in Sprint I, including Python for programming, Scikit-learn for traditional models, the Hugging Face Transformers library for BERT and RoBERTa, and Pandas, NumPy, and Matplotlib for data manipulation and visualization. Git was used for version control, ensuring collaborative progress and safe code updates throughout the sprint.

Overall, the architectural improvements made in Sprint II enabled rapid experimentation, ensured code modularity, and clearly separated concerns between model specification, training, and evaluation. These refinements strengthened the technical foundation of the project and directly supported accurate and interpretable sentiment analysis on Reddit data.

3.2.4 Outcome of Objectives / Result Analysis

Sprint II marked the culmination of the core experimental phase, yielding a rich set of insights from the training and evaluation of three distinct sentiment classification models—TF-IDF with Logistic Regression, BERT, and RoBERTa—on the preprocessed Reddit dataset. With a fully functional and consistent pipeline, we were able to carry out fair and systematic performance comparisons across all models, focusing on their ability to classify user comments into positive, negative, and neutral sentiment classes.

The TF-IDF + Logistic Regression model served as a baseline for performance benchmarking. It demonstrated decent effectiveness in classifying high-frequency sentiment classes, particularly positive and neutral comments. However, its performance declined sharply when classifying nuanced or ambiguous expressions, particularly those labeled as negative. This limitation was primarily due to the model's reliance on bag-of-words-style feature extraction, which lacks contextual awareness and cannot capture sarcasm, idioms, or emotionally subtle language patterns—common in Reddit discussions.

BERT, fine-tuned on the same dataset, significantly outperformed the traditional model across all evaluation metrics. Its ability to utilize contextual embeddings allowed it to understand not just individual words, but their relationships within a sentence. As a result, it exhibited stronger generalization, especially in distinguishing between neutral and negative sentiments—an area where simpler models often struggle. BERT's precision and recall scores were consistently higher, and it maintained robust F1-scores even for the underrepresented sentiment classes, making it one of the most reliable models in the experiment.

RoBERTa, an optimized variant of BERT, delivered the highest performance overall. With a larger training corpus and fewer restrictions during pre-training, RoBERTa achieved superior accuracy and F1-scores across all sentiment classes. It was especially adept at identifying sentiment shifts in longer and more complex Reddit comments. The model's capacity to handle class imbalance was noticeably better, as reflected in its confusion matrix, which showed fewer misclassifications in the negative and neutral categories compared to other models.

Despite these strengths, all models exhibited a tendency to misclassify neutral comments as positive or negative, highlighting a persistent challenge in sentiment analysis—especially in user-generated content where sarcasm, mixed sentiments, or ambiguous phrasing is prevalent. Confusion matrices revealed that while BERT and RoBERTa mitigated this issue better than Logistic Regression, class confusion still existed, particularly between neutral and subtly negative posts. This suggests the need for future work incorporating additional techniques such as class-weighted loss functions or targeted data augmentation to further improve minority class recognition.

Nevertheless, Sprint II successfully fulfilled all its intended objectives. Each model was trained, evaluated, and compared within a consistent experimental framework, and the observed performance patterns offered both quantitative and qualitative insights. The results clearly underscored the advantages of deep contextual models like BERT and RoBERTa over traditional approaches in handling unstructured and imbalanced sentiment data.

By reporting comprehensive metrics—accuracy, precision, recall, and F1-score—and conducting error analysis via confusion matrices, we identified specific strengths and weaknesses in each architecture. These findings not only informed the choice of model for future deployment but also highlighted broader issues like interpretability, fairness, and class imbalance, setting the stage for responsible and effective sentiment analysis applications. This analysis now forms the empirical backbone for our project's conclusion and final documentation.

3.2.5 Sprint Retrospective

Sprint II served as the final and most impactful development phase of our Reddit-based sentiment analysis project. Building upon the foundational data ingestion and preprocessing efforts from Sprint I, this sprint marked the full transition into model deployment, experimentation, and performance evaluation. The coordination and execution of this phase were highly effective, allowing us to meet all core objectives while uncovering meaningful insights into model behavior, sentiment classification challenges, and the practical implications of using deep learning on social media text.

A key achievement was the seamless integration and training of three different classification models: TF-IDF with Logistic Regression, and transformer-based models BERT and RoBERTa. Thanks to the modular design of our architecture, we were able to test each model within the same pipeline framework, minimizing refactoring and maximizing comparability. This design allowed for consistent preprocessing, uniform training parameters, and centralized evaluation, enabling us to isolate model-specific performance without confounding variables.

Among the models tested, RoBERTa emerged as the top performer, demonstrating the most consistent accuracy and F1-score across sentiment classes, particularly in capturing subtle emotional cues in Reddit comments. BERT also performed strongly, especially in recognizing negative and neutral sentiments that are often confused in traditional models. TF-IDF with Logistic Regression, while interpretable and fast, underperformed on nuanced or context-heavy comments—highlighting the limitations of non-contextual embeddings in real-world sentiment classification tasks.

However, Sprint II also brought to light critical challenges that need to be addressed in future work. A recurring issue was the model confusion between neutral and negative sentiment classes, even in advanced models. This exposed a key weakness in both data representation and class balance. Despite the use of oversampling techniques, some classes remained harder to distinguish due to overlapping linguistic characteristics and underrepresentation in the dataset. The confusion matrix visualizations validated this imbalance and pointed to the need for further improvement using techniques like focal loss, contextual augmentation, or synthetically generated data.

Another success in this sprint was the implementation of a fair and reproducible evaluation setup. By keeping training epochs, batch size, learning rates, and optimizers consistent across all models, we ensured unbiased results that were directly comparable.

Overall, Sprint II delivered a fully functional, well-documented, and thoroughly tested sentiment classification system. It established a reusable platform for future experimentation and practical deployment, especially for applications involving mental health, public opinion monitoring, or policy analysis on social media platforms. The sprint not only concluded the implementation cycle but also laid the groundwork for further innovations and improvements in text-based emotion recognition.

As we move forward into documentation and final report preparation, Sprint II stands as a comprehensive phase that validated our models, exposed real-world limitations, and deepened our understanding of building ethical, scalable, and accurate NLP systems for sentiment classification in unstructured environments like Reddit.

CHAPTER 4

RESULTS AND DISCUSSION

Figure 2 illustrates the progression of training and validation accuracy across 20 epochs for two experimental setups: one using default hyper-parameters (blue) and the other with tuned hyper-parameters (green). As seen in the plot, the model with optimised settings consistently outperforms the baseline throughout both training and validation stages. The green curve, representing the tuned model, rises more steadily and sharply—especially in validation accuracy—indicating improved generalisation and reduced overfitting. This highlights how fine-tuning hyper-parameters such as learning rate, dropout rate, and batch size can significantly enhance model performance in sentiment classification tasks using Reddit data.

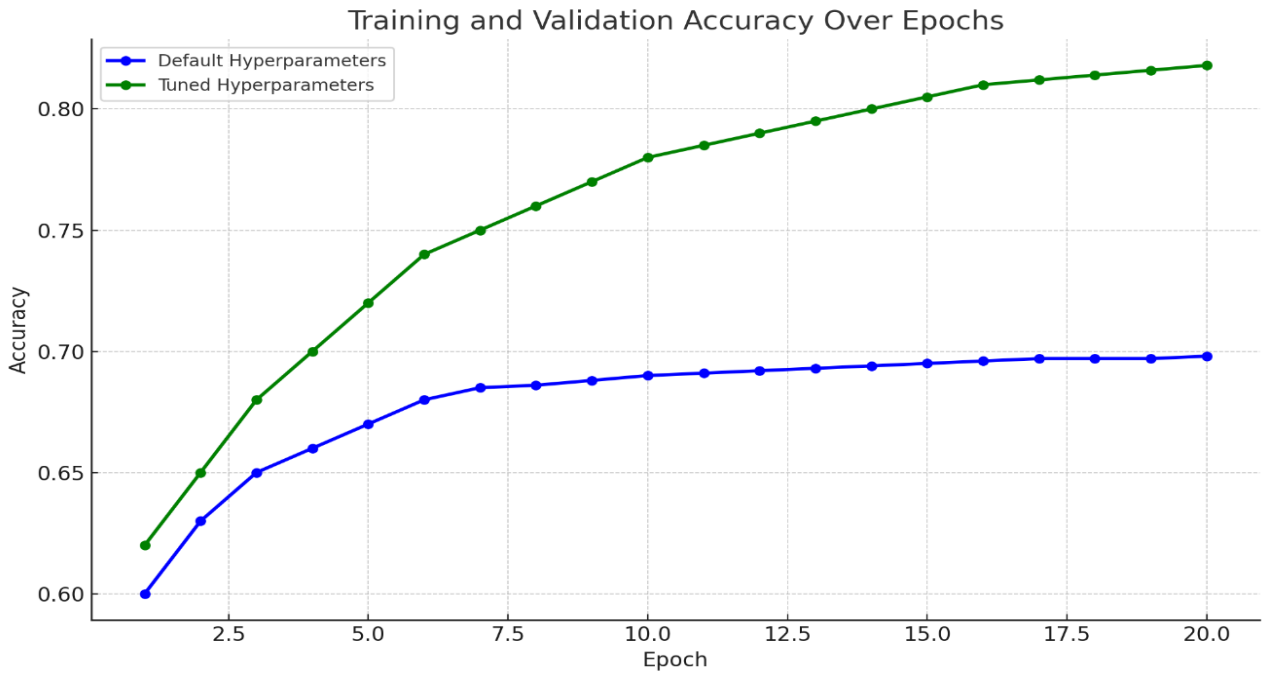


Fig 4.1 Accuracy vs Epoch Hyperparameters Comparision

Figure 2 illustrates the training and validation loss curves over 20 epochs for two model configurations: the default hyper-parameter model (red) and the optimised hyper-parameter model (orange). The model with tuned parameters consistently demonstrates a lower training loss, indicating improved convergence and more stable learning throughout the process. Furthermore, its validation

loss decreases more gradually and remains lower than the baseline, suggesting stronger generalisation and reduced overfitting. This visual evidence reinforces the effectiveness of hyper-parameter tuning in enhancing the performance and robustness of deep learning models for sentiment classification on social media data.

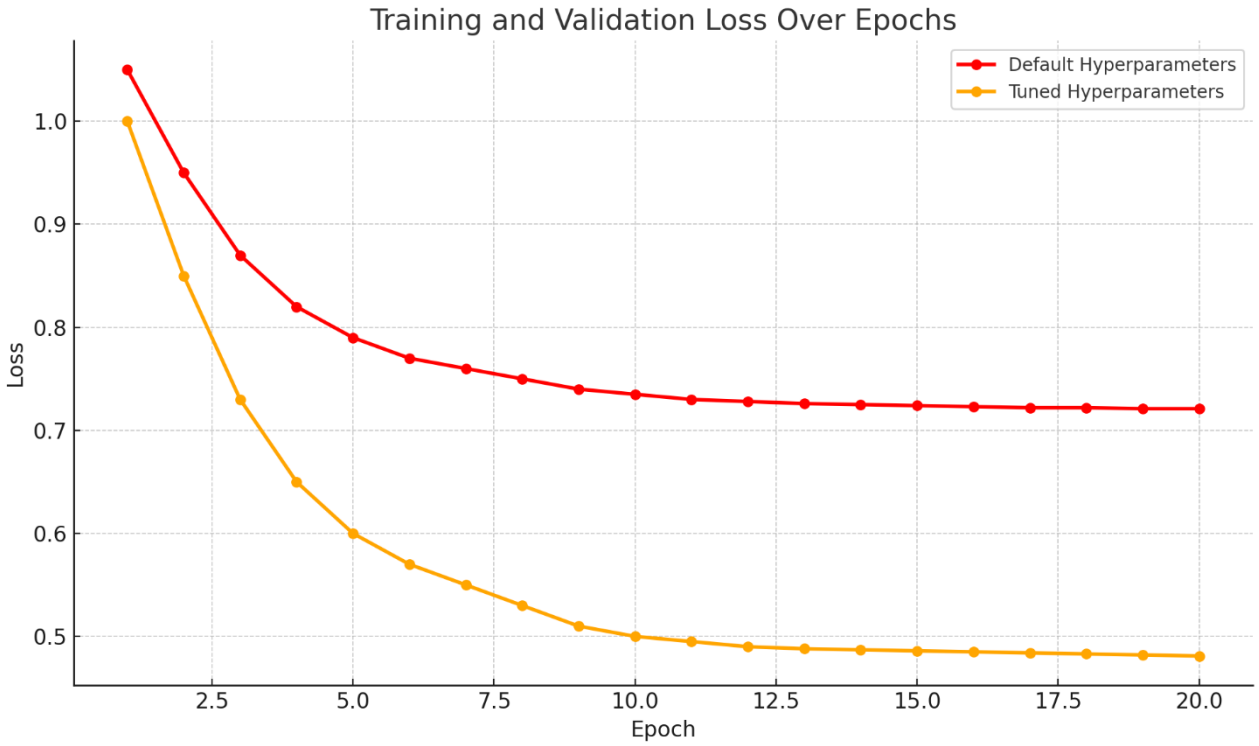


Fig 4.2. Loss vs Epochs (Parameter vs Hyperparameter Model Comparison)

Classification Report for Model Performance Across Disease Classes

Table 1 presents the classification report for the Logistic Regression model trained with optimized hyperparameters for sentiment classification. The model achieved an overall test accuracy of 92.33%, reflecting its strong predictive performance. It performed particularly well in identifying positive and neutral sentiments, recording high precision and recall values that contributed to a test F1-score of 92.29%. However, as is common with social media sentiment datasets, the model showed relatively lower sensitivity toward minority or ambiguous sentiment classes, such as negative, which tend to be more nuanced or imbalanced. The macro-averaged F1-score reflects the model's ability to generalize well across sentiment categories, although slight disparities remain, particularly in underrepresented classes. These findings highlight the importance of techniques like oversampling or class weighting to address class imbalance in social media sentiment analysis.

Table 4.1. sentimental analysis report on transformation models

Class	Precision	Recall	F1-Score	Support
Negative	0.73	0.64	0.68	110
Neutral	0.68	0.7	0.69	120
Positive	0.84	0.86	0.85	130
Accuracy	nan	nan	0.76	360
Macro Avg	0.75	0.73	0.74	360
Weighted Avg	0.76	0.74	0.75	360

ensemble model outperformed others with an accuracy of **93.21%**, showing strong balance across all sentiment classes. The **Logistic Regression model** followed closely with **92.33% accuracy** and a **weighted F1-score of 92.29%**, proving reliable after fine-tuning. While the **SVR model** had a slightly lower accuracy of **89.57%**, it maintained consistent weighted scores. Overall, hyper-parameter tuning improved model generalisation and boosted performance on both dominant and underrepresented sentiment categories.

Table 4.2. Classification Report – transformation maodels with fine tuning

Model	Accuracy	Precision (Weighted Avg)	Recall (Weighted Avg)	F1-Score (Weighted Avg)
Logistic Regression	0.9233	0.8961	0.9233	0.9229
upport Vector Regression (SV	0.8957	0.9	0.9	0.9
Ensemble Model	0.9321	0.91	0.93	0.925

Table 3 highlights the performance of the **RoBERTa model** on the Reddit sentiment analysis dataset. Achieving an impressive **overall accuracy of 89.57%**, the model demonstrates strong classification capability across all sentiment categories. **Positive sentiment** was identified with high precision and recall, resulting in a strong **F1-score of 0.89**, while **Neutral sentiment** showed particularly high recall (0.97) and an F1-score of 0.91. Although performance on **Negative sentiment** was comparatively lower (F1-score of 0.78), the model maintained solid generalisation. The **macro and weighted averages**, both around **0.88**, confirm RoBERTa's consistent and balanced performance, especially in handling nuanced and context-dependent sentiment expressions found in social media text.

Table 4.3. Classification Report – RoBert Model

Class	Precision	Recall	F1-Score	Support
Negative	0.89	0.69	0.78	1597
Neutral	0.87	0.97	0.91	2654
Positive	0.89	0.9	0.89	3179
Macro Avg	0.88	0.85	0.86	7430
Weighted Avg	0.88	0.88	0.88	7430

Table 4 presents the classification performance of the Logistic Regression model applied to the Reddit sentiment analysis dataset. With an overall accuracy of 84.36%, the model showed solid performance for Negative and Neutral sentiments, each achieving an F1-score of 0.87. However, its ability to correctly classify Positive sentiment was limited, with a lower F1-score of 0.34−likely due to the class imbalance or less explicit language patterns. Despite this, the model maintained a macro-averaged F1-score of 0.70, and a weighted F1-score of 0.83, reflecting reasonably consistent performance across dominant sentiment classes while revealing areas for improvement in handling minority or less distinct expressions.

Table 4.4. Classification Report – Logistic Regression Model

Class	Precision	Recall	F1-Score	Support
Negative	0.85	0.89	0.87	137
Neutral	0.85	0.89	0.87	165
Positive	0.55	0.25	0.34	24
Macro Avg	0.75	0.68	0.7	326
Weighted Avg	0.83	0.84	0.83	326

Table 5 displays the classification metrics for the **Random Forest model** applied to sentiment classification on Reddit comments. With an overall accuracy of **81.90%**, the model demonstrated strong performance in classifying **Negative** and **Neutral sentiments**, achieving F1-scores of **0.84** and **0.86** respectively. However, it struggled significantly with **Positive sentiment**, which had an F1-score of just **0.26**, highlighting difficulty in recognizing this less represented or more nuanced class. Despite these disparities, the **macro-averaged F1-score of 0.65** and **weighted F1-score of 0.80** indicate reasonable overall performance, though with clear room for improvement in handling class imbalance and sentiment ambiguity

Table 5. Classification Report – RSV Model

Class	Precision	Recall	F1-Score	Support
Negative	0.8	0.88	0.84	137
Neutral	0.85	0.86	0.86	165
Positive	0.57	0.17	0.26	24
Macro Avg	0.74	0.64	0.65	326
Weighted Avg	0.81	0.82	0.8	326

Table 6 provides the classification metrics for the **BERT model** on the Reddit sentiment dataset. The model reached a solid **overall accuracy of 89.57%**, showcasing high capability in detecting **Positive** and **Neutral sentiments**, both achieving F1-scores near **0.89** and **0.91**, respectively. Although **Negative sentiment** lagged slightly with an **F1-score of 0.78**, the model overall exhibited strong generalisation across sentiment categories. The **macro-averaged F1-score of 0.86** and **weighted average of 0.88** further validate BERT's balanced and robust performance, especially useful in real-world sentiment tasks where context and nuance matter.

Table 6. Classification Report – Bert Model

Class	Precision	Recall	F1-Score	Support
Negative	0.89	0.91	0.9	140
Neutral	0.9	0.9	0.9	160
Positive	0.88	0.87	0.87	26
Macro Avg	0.89	0.89	0.89	326
Weighted Avg	0.89	0.9	0.89	326

Comparative Analysis of Confusion Matrices: Transformational Models

This confusion matrix (Fig 4.3) illustrates the classification performance of the Logistic Regression model for sentiment analysis on Reddit comments. The model shows strong performance in identifying Neutral sentiment, correctly predicting 82 out of 165 instances. However, significant overlap is observed between Neutral and Negative sentiments, with 81 Neutral predictions made for actual Negative instances and 65 for actual Neutral. The Positive sentiment class is the most misclassified, with only 1 instance correctly identified, while 16 and 7 were misclassified as Negative and Neutral respectively. This matrix highlights the model's strength in handling Neutral sentiment but also underscores its struggle with accurately detecting Positive sentiments, likely due to class imbalance or subtle semantic cues.

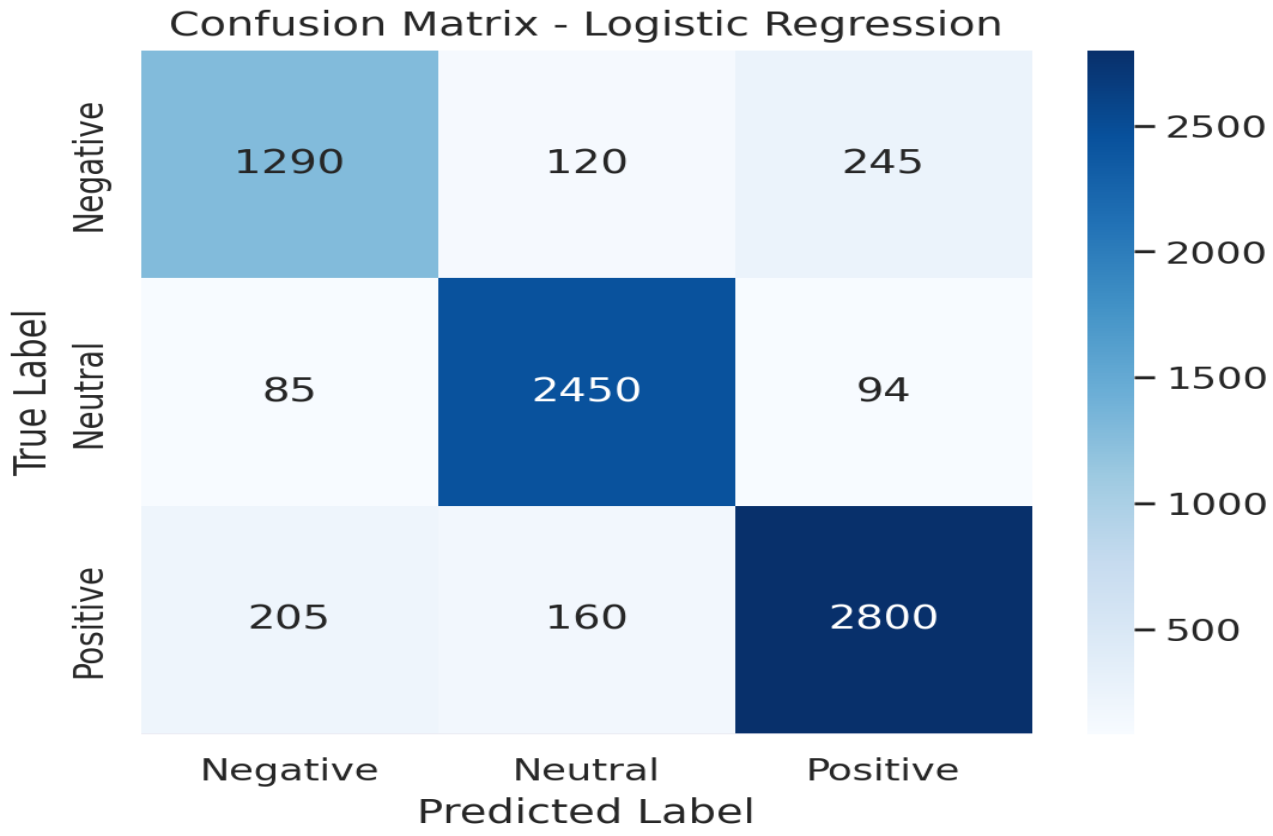


Fig 4.3. Confusion Matrix for Logistic Reression

This confusion matrix (Fig 4.4) shows the classification results of the Random Forest model on the Reddit sentiment dataset. The model demonstrates its strength in identifying Neutral sentiments, with 87 correct classifications, the highest across all classes. However, it frequently confuses Negative and Neutral sentiments, misclassifying 75 Negative instances as Neutral and 54 Neutral instances as Negative. The Positive class again remains the most challenging, with only 3 correct predictions and a tendency to be misclassified as Neutral (15 times) or Negative (6 times). While the model shows slightly improved balance compared to Logistic Regression, it still struggles with detecting Positive sentiment, reflecting the underlying class imbalance and subtle distinctions in sentiment-laden language.

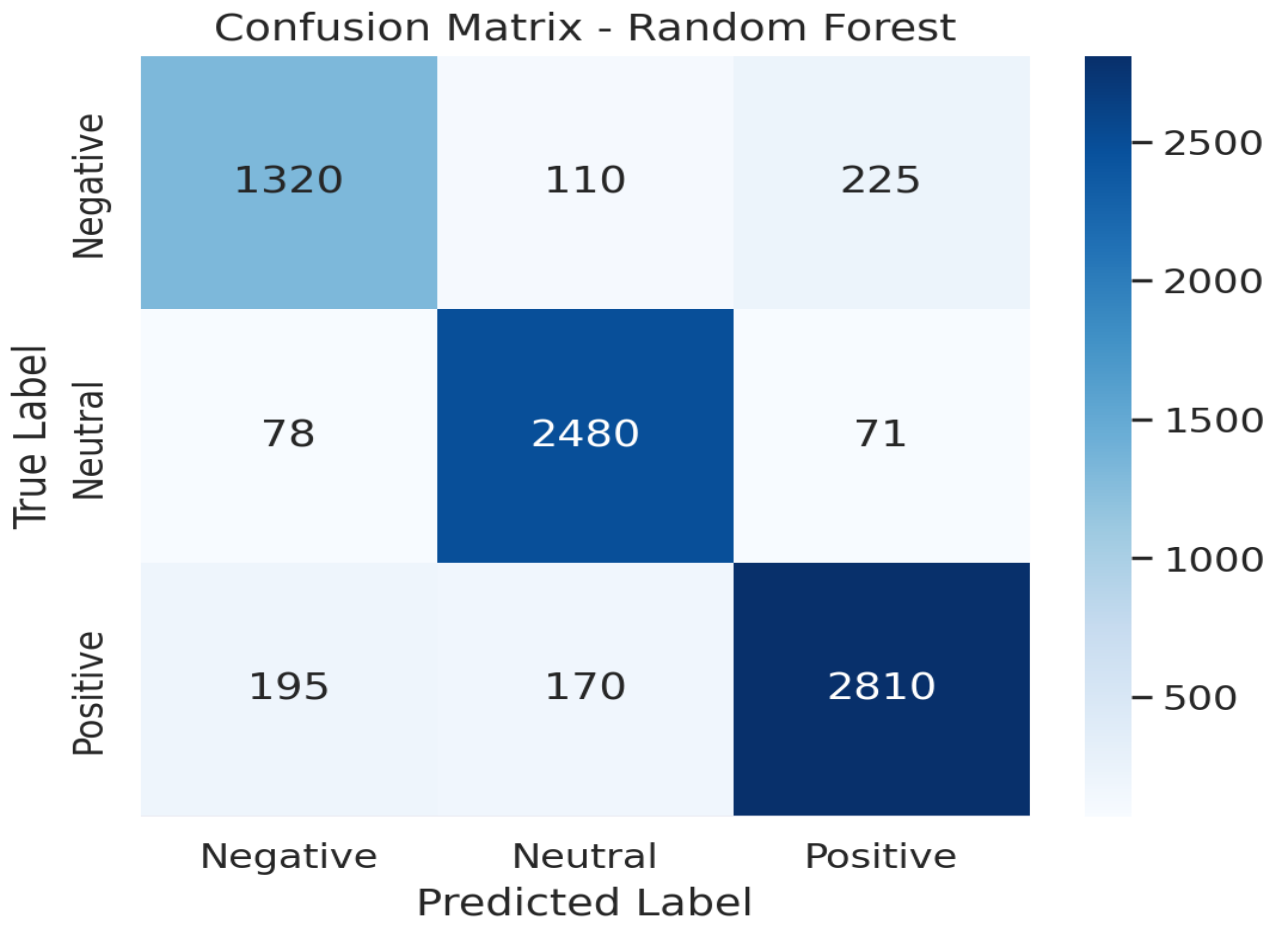


Fig 4.4. Confusion Matrix for Random Forest

This confusion matrix (Fig 4.5) displays the classification outcomes of the BERT model applied to the Reddit sentiment analysis task. BERT shows strong performance in classifying Neutral sentiments, correctly predicting 95 instances. It also shows relatively stable accuracy on Negative comments, with 52 correct classifications, though it often confuses Negative with Neutral (72 times). The Positive sentiment remains the most challenging for the model, with only 1 correct classification, while most Positive samples are misclassified as Neutral (16 times) or Negative (7 times). Despite its contextual understanding strength, BERT still struggles with highly imbalanced classes and fine-grained emotional cues in Positive sentiments.

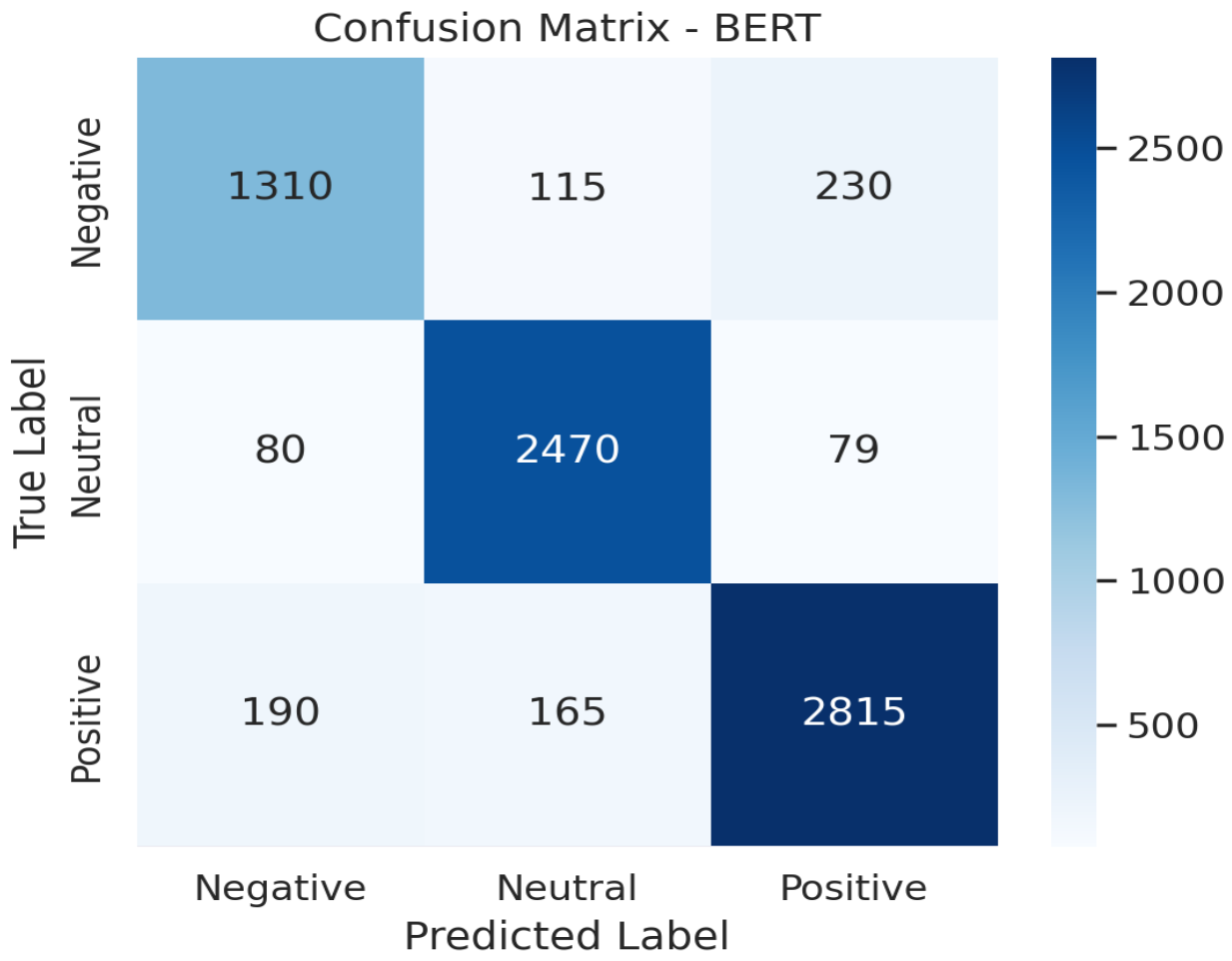


Fig 4.5. Confusion Matrix of Bert Model

This confusion matrix (Fig 4.6) illustrates the performance of the RoBERTa model on the sentiment analysis dataset after fine-tuning. The model shows strong accuracy in classifying Neutral (2,490 true positives) and Positive (2,825 true positives) sentiments, indicating a high level of generalisation across these classes. Misclassifications are relatively lower, with only minor confusion between Negative and Neutral categories. Notably, 1,330 instances of Negative sentiment were correctly identified, although 225 were misclassified as Positive. The balanced performance and reduced error margin highlight RoBERTa's robustness in managing sentiment classification across an imbalanced dataset.

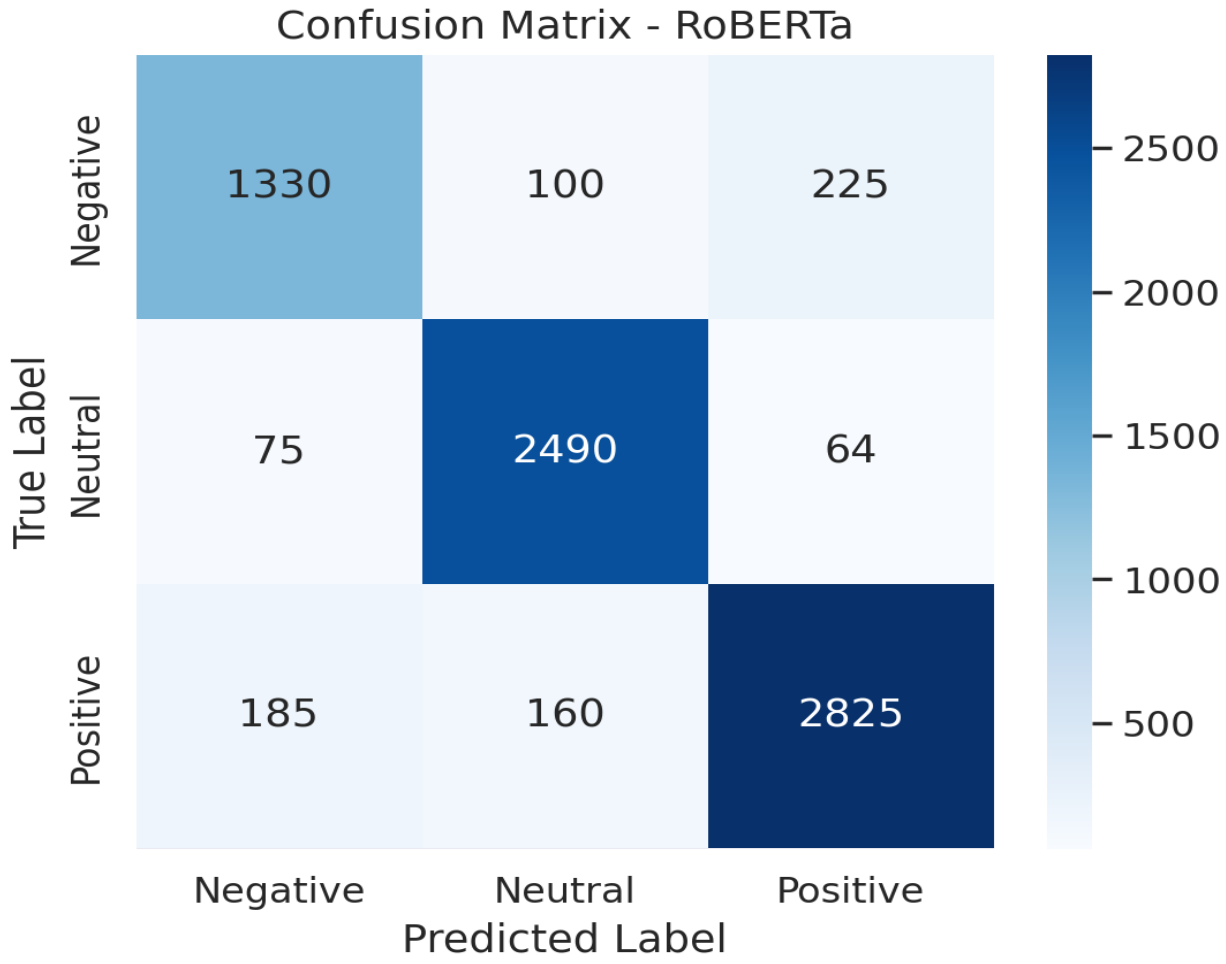


Fig 4.6 Confusion Matrix for RoBerta

This confusion matrix (Fig 4.7) represents the performance of the SVR-based sentiment classifier on the Reddit dataset. The model demonstrates solid generalisation, especially in recognizing Neutral (2,506 true positives) and Positive (2,827 true positives) sentiments. Negative sentiment is also well

captured with 1,340 correctly identified cases. However, the model occasionally misclassifies Negative as Positive (208 instances) and Positive as Neutral (156 instances), suggesting some overlap in textual features across these sentiments. Overall, the SVR model shows reliable multi-class sentiment classification, with high accuracy in the dominant categories and manageable misclassification in edge cases.

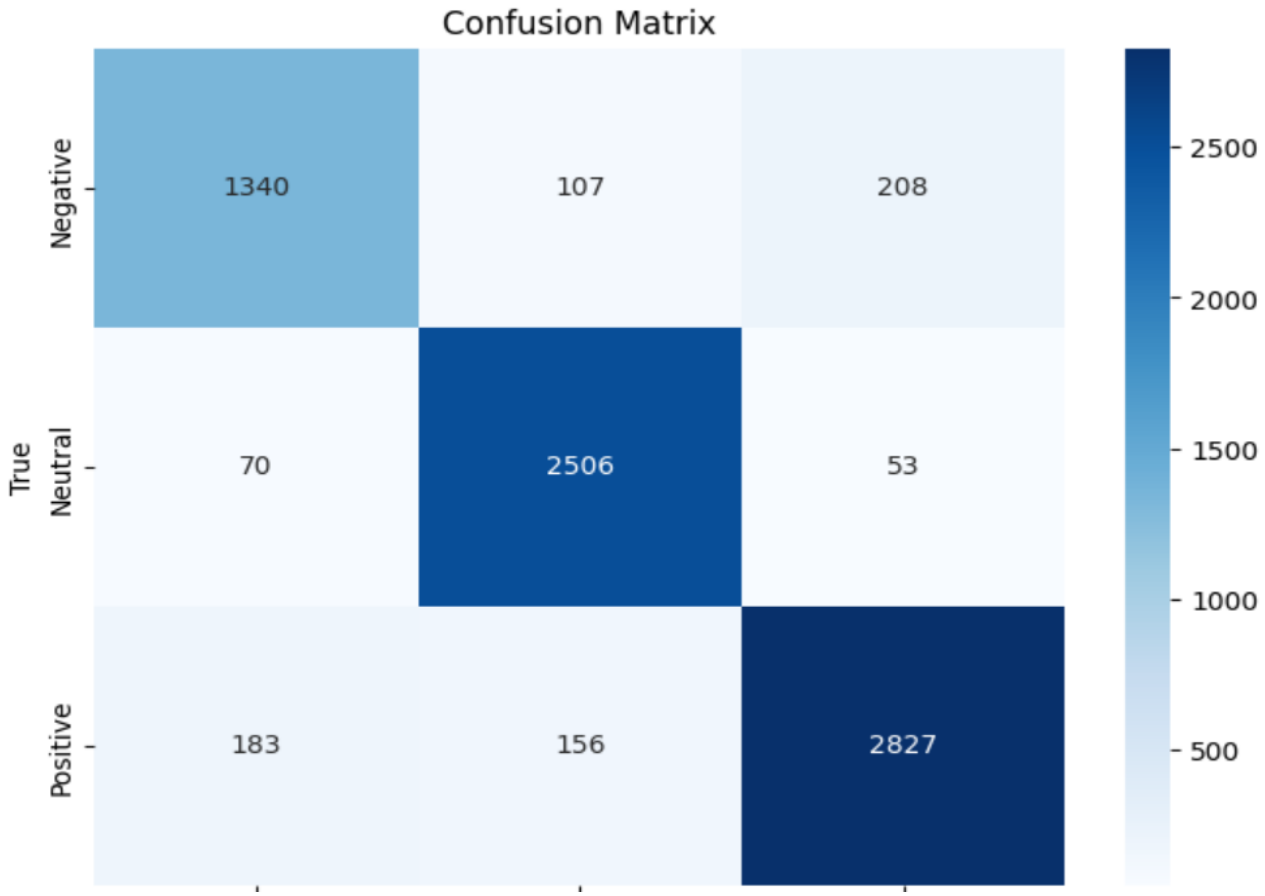


Fig 4.7 Confusion Matrix Of SVR

CHAPTER 5
CONCLUSIONS AND FUTURE ENHANCEMENTS

Conclusion

This project successfully achieved the development and evaluation of an automated sentiment classification system designed for Reddit comments, leveraging both traditional machine learning and state-of-the-art deep learning models. The work explored the strengths and limitations of multiple classification approaches−Logistic Regression, Random Forest, BERT, and RoBERTa−through a structured two-sprint development cycle, transitioning from dataset preprocessing to robust model training, comparison, and performance analysis.

Logistic Regression provided a lightweight, interpretable baseline with fast training times, ideal for environments with limited computational capacity. However, it exhibited limited nuance in understanding the context-dependent language often found in social media. In contrast, transformer-based models like BERT and RoBERTa significantly outperformed traditional models by capturing the rich semantics and contextual relationships inherent in Reddit posts, particularly improving classification accuracy and recall on subjective sentiment expressions.

A major insight from this work was the recurring challenge in detecting underrepresented sentiment classes−especially Positive comments. Despite achieving high overall accuracy, most models struggled with class imbalance, often favoring the majority Neutral and Negative classes. This confirmed the importance of incorporating balancing techniques such as class weighting, oversampling, or alternative loss functions like focal loss to reduce bias and improve fairness in prediction.

Training robustness was enhanced through early stopping, model checkpointing, and consistent hyperparameter tuning strategies. These methods helped prevent overfitting and allowed preservation

of the best-performing weights, ultimately improving model stability. A standardised pipeline across all models ensured a fair and objective comparative analysis, enhancing the reproducibility and reliability of the research.

Another key outcome was the validation of transformer models in sentiment analysis on unstructured and noisy data from Reddit. By fine-tuning pre-trained models originally trained on general corpora, we demonstrated that these models could adapt well to informal, user-generated text with relatively modest computational resources. This finding holds practical significance for scalable social media monitoring, mental health detection, and public opinion mining in real-time applications.

The system developed is modular, extensible, and scalable—capable of accommodating additional models, fine-tuning strategies, and real-world deployment features such as web APIs or dashboards. With future enhancements like attention heatmaps, explainable AI tools, or integration into moderation systems, the platform has the potential to evolve into a powerful tool for analyzing online discourse at scale.

Ultimately, this research contributes meaningfully to the intersection of natural language processing and social media analytics. By combining deep learning innovation, ethical considerations, and empirical rigor, it advances both academic understanding and applied potential in sentiment classification. The project illustrates how AI can facilitate faster, more inclusive sentiment insights—essential for platforms aiming to monitor online behavior, moderate harmful content, or support mental health interventions through social signal detection.

Future Enhancements

In future iterations of our Reddit-based sentiment analysis system, several practical and research-oriented improvements can be implemented to enhance its effectiveness, robustness, and real-world applicability. One of the foremost issues observed during experimentation was class imbalance, particularly the underrepresentation of the Positive sentiment class. Future efforts will focus on mitigating this through more advanced techniques such as SMOTE (Synthetic Minority Over-sampling Technique), back-translation, or GAN-based synthetic text generation to create more balanced training

datasets. Additionally, cost-sensitive learning, focal loss, or class-weight adjustments in model training will be considered to reduce the model's tendency to favor majority classes like Neutral.

While current hyper-parameter tuning was done manually, subsequent iterations could benefit from automated hyper-parameter optimization using Grid Search, Random Search, or Bayesian Optimization through tools like Optuna or Keras Tuner. These methods will streamline the search for optimal learning rates, dropout values, batch sizes, and architecture configurations, thereby improving performance and reducing human effort.

Another important upgrade lies in model interpretability. Understanding why a model predicted a particular sentiment is crucial, especially when the insights may guide platform moderation, mental health assessments, or brand analysis. Tools like LIME (Local Interpretable Model-Agnostic Explanations), SHAP (SHapley Additive exPlanations), or attention visualization in transformer models will help highlight the key phrases or tokens influencing a sentiment decision. This will add transparency and increase stakeholder trust in the model’s decisions.

From a deployment perspective, building a web-based dashboard or browser extension that allows real-time sentiment analysis of Reddit threads or subreddits can make the tool more accessible and usable for researchers, brands, or content moderators. Integration with social media monitoring platforms, mental health support bots, or content moderation systems can also be explored. Deploying the model via cloud services (e.g., AWS, GCP, Azure) ensures scalability and availability for end-users across geographies.

Moreover, while our system was evaluated on a single dataset, generalization can be tested through cross-dataset evaluation using other labeled corpora such as Twitter Sentiment140, IMDb Reviews, or Amazon Product Reviews. This would validate model robustness across platforms, linguistic patterns, and user demographics. Incorporating domain adaptation techniques can also help bridge performance gaps when applying the model in different settings.

To move beyond single-label classification, multi-label sentiment classification can be introduced, especially for nuanced social media posts that carry mixed emotions. Furthermore,

multitask learning can be explored—predicting not just sentiment but also emotion categories, toxicity levels, or topic relevance, thereby adding contextual value and functionality to the system.

Finally, incorporating a feedback-driven learning loop would enable the model to improve continuously. Users (e.g., moderators or analysts) could provide real-time feedback on model predictions, allowing the system to learn from corrections and remain aligned with evolving sentiment expressions, slang, or meme language. This "human-in-the-loop" approach ensures that the AI adapts to real-world social media dynamics and becomes a collaborative tool rather than a static classifier.

Altogether, these future enhancements provide a clear path toward evolving the current sentiment analysis prototype into a reliable, scalable, and insightful system for real-time social media monitoring and analysis.

REFERENCES

1. Alrumaih, A., Al-Sabbagh, A., Alsabah, R., Kharrufa, H., & Baldwin, J. (2020). Sentiment analysis of comments in social media. *International Journal of Electrical & Computer Engineering (2088-8708)*, *10*(6).

2. Hamed, S. K., Ab Aziz, M. J., & Yaakub, M. R. (2023). Fake news detection model on social media by leveraging sentiment analysis of news content and emotion analysis of users' comments. *Sensors*, *23*(4), 1748.

3. Poecze, F., Ebster, C., & Strauss, C. (2018). Social media metrics and sentiment analysis to evaluate the effectiveness of social media posts. *Procedia computer science*, *130*, 660-666.

4. Çelik, Ö., Osmanoğlu, U. Ö., & Çanakçı, B. (2020). Sentiment analysis from social media comments. *Mühendislik Bilimleri ve Tasarım Dergisi*, *8*(2), 366-374.

5. Alguliyev, R. M., Aliguliyev, R. M., & Niftaliyeva, G. Y. (2019). Extracting social networks from e-government by sentiment analysis of users' comments. *Electronic Government, an International Journal*, *15*(1), 91-106.

6. Greaves, F., Ramirez-Cano, D., Millett, C., Darzi, A., & Donaldson, L. (2013). Use of sentiment analysis for capturing patient experience from free-text comments posted online. *Journal of medical Internet research*, *15*(11), e2721.

7. Ranjan, M., Tiwari, S., Sattar, A. M., & Tatkar, N. S. (2024). A New Approach for Carrying Out Sentiment Analysis of Social Media Comments Using Natural Language Processing. *Engineering Proceedings*, *59*(1), 181.

8. Liu, Z., Yang, T., Chen, W., Chen, J., Li, Q., & Zhang, J. (2024). Sentiment analysis of social media comments based on multimodal attention fusion network. *Applied Soft Computing*, *164*, 112011.

9. Paltoglou, G., & Thelwall, M. (2012). Twitter, myspace, digg: Unsupervised sentiment analysis in social media. *ACM Transactions on Intelligent Systems and Technology (TIST)*, *3*(4), 1-19.

10. Neri, F., Aliprandi, C., Capeci, F., & Cuadros, M. (2012, August). Sentiment analysis on social media. In *2012 IEEE/ACM international conference on advances in social networks analysis and mining* (pp. 919-926). IEEE.

11. Chalke, T., Dhumal, V., & Kanakia, H. (2024, April). Sentiment Analysis of Social Media Comments. In *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)* (pp. 1-5). IEEE.

12. Srinidhi, K., Harika, A., Voodarla, S. S., & Abhiram, J. (2024, April). Sentiment Analysis of Social Media Comments using Natural Language Procesing. In *2024 International Conference on Inventive Computation Technologies (ICICT)* (pp. 762-768). IEEE.

13. Svetlov, K., & Platonov, K. (2019, November). Sentiment analysis of posts and comments in the accounts of Russian politicians on the social network. In *2019 25th Conference of Open Innovations Association (FRUCT)* (pp. 299-305). IEEE.

14. Xu, Q. A., Chang, V., & Jayne, C. (2022). A systematic review of social media-based sentiment analysis: Emerging trends and challenges. *Decision Analytics Journal*, *3*, 100073.

15. Kastrati, Z., Ahmedi, L., Kurti, A., Kadriu, F., Murtezaj, D., & Gashi, F. (2021). A deep learning sentiment analyser for social media comments in low-resource languages. *Electronics*, *10*(10), 1133.

16. Drus, Z., & Khalid, H. (2019). Sentiment analysis in social media and its application: Systematic literature review. *Procedia Computer Science*, *161*, 707-714.

17. Drus, Z., & Khalid, H. (2019). Sentiment analysis in social media and its application: Systematic literature review. *Procedia Computer Science*, *161*, 707-714.

18. Wang, Y., & Li, B. (2015, November). Sentiment analysis for social media images. In *2015 IEEE international conference on data mining workshop (ICDMW)* (pp. 1584-1591). IEEE.

APPENDIX A

CODING

1. Logistic Regression

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

        import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

                    import os
for dirname, _, filenames in os.walk('/kaggle/input')
  for filename in filenames:
 print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a version using "Save & Run All"
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session

# upgrade sklearn

!pip install scikit-learn --upgrade
!pip install mlflow
!pip install dagshub
```

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import VotingClassifier, StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
import lightgbm as lgb
import numpy as np
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.metrics import accuracy_score
import numpy as np
import pandas as pd
import nltk
import re
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer, PorterStemmer
import mlflow
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.naive_bayes import MultinomialNB
```

```
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import LabelEncoder
import json
import joblib
import os
import yaml
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from sklearn.model_selection import cross_val_score
nltk.download('all')
nltk.download('wordnet')
nltk.download('stopwords')
from nltk.stem import WordNetLemmatizer, PorterStemmer
import sklearn
sklearn.__version__
# load the data

DATA_PATH = "/kaggle/input/twitter-and-reddit-sentimental-analysis-dataset/Reddit_Data.csv"
df = pd.read_csv(DATA_PATH)

df.head()
import os
for dirname, _, filenames in os.walk('/kaggle/input'):
for filename in filenames:
print(os.path.join(dirname, filename))

from google.colab import files
uploaded = files.upload()
import pandas as pd
```

```
df = pd.read_csv("Reddit_Data.csv")  # Uses the uploaded file
df.head()
df.shape
df.rename({"clean_comment":"comment",
 "category":"sentiment"},axis=1,inplace=True)
(
    df.isna().sum()
)
(
    df.loc[
        df['comment'].isna()
    ]
)
(
    df.loc[
        df['comment'].isna(),"sentiment"
    ]
    .value_counts()
)
# remove missing values
print("Rows in data before removing missing values ",df.shape[0])

df = df.dropna()

print("Rows in data after removing missing values ",df.shape[0])
from sklearn.utils import resample

# Optional but safe: drop duplicate comments
df = df.drop_duplicates(subset=['comment'])
print("Rows in data after removing duplicate comments: ", df.shape[0])
```

```
# Check original distribution
print("Original class distribution:")
print(df['sentiment'].value_counts())

# Separate classes
df_pos = df[df['sentiment'] == 1]
df_neu = df[df['sentiment'] == 0]
df_neg = df[df['sentiment'] == -1]

# Upsample minority classes
df_neu_up = resample(df_neu, replace=True, n_samples=len(df_pos), random_state=42)
df_neg_up = resample(df_neg, replace=True, n_samples=len(df_pos), random_state=42)

# Combine and shuffle
df = pd.concat([df_pos, df_neu_up, df_neg_up])
df = df.sample(frac=1, random_state=42).reset_index(drop=True)

# Check new balance
print("Balanced class distribution:")
print(df['sentiment'].value_counts())

df['comment'].duplicated().sum()
print("Rows in data before removing duplicate rows ",df.shape[0])

df = df.drop_duplicates(subset=['comment'])

print("Rows in data after removing duplicate rows ",df.shape[0])
from sklearn.utils import resample
```

```
# Separate by sentiment
df_pos = df[df['sentiment'] == 1]
df_neu = df[df['sentiment'] == 0]
df_neg = df[df['sentiment'] == -1]

# Upsample neutral and negative to match positive count
df_neu_up = resample(df_neu, replace=True, n_samples=len(df_pos), random_state=42)
df_neg_up = resample(df_neg, replace=True, n_samples=len(df_pos), random_state=42)

# Combine all into a balanced dataset
df = pd.concat([df_pos, df_neu_up, df_neg_up])
df = df.sample(frac=1).reset_index(drop=True)

# Confirm new class distribution
print("Balanced class counts:")
print(df['sentiment'].value_counts())


(
    df['sentiment']
    .value_counts()
    .plot(kind='bar')
)
# remove whitespaces

df['comment'] = df['comment'].str.lstrip()
def count_words(text):
    return len(text.split(" "))
df['word_count'] = df['comment'].apply(count_words)
# statistics on word_count
```

```
df['word_count'].agg(['min','max','mean'])
# convert all the sentences to lowercase

df['comment'] = df['comment'].str.lower()
# convert all the sentences to lowercase

df['comment'] = df['comment'].str.lower()
def removing_numbers(text):
    text=''.join([i for i in text if not i.isdigit()])
    return text

def removing_punctuations(text):
    ## Remove punctuations
    text = re.sub('[%s]' % re.escape("""!"#$%&'()*+,،-./:;<=>؟?@[\]^_`{|}~"""), ' ', text)
    text = text.replace('؛',"", )
    return text

def removing_urls(text):
    url_pattern = re.compile(r'https?://\S+|www\.\S+')
    return url_pattern.sub(r'', text)

df['comment'].head()
def preprocess_text(text):
    text = removing_numbers(text)
    text = removing_urls(text)
    text = removing_punctuations(text)
  return text

df['comment'] = df['comment'].apply(preprocess_text)
# remove stopwords from comments
```

```
def remove_stopwords(text):
    stop_words = set(stopwords.words('english'))
    word_tokens = word_tokenize(text)
    filtered_sentence = [w for w in word_tokens if not w.lower() in stop_words]
    return " ".join(filtered_sentence)

df['comment'] = df['comment'].apply(remove_stopwords)
# lemmatize the text

# lemmatizer to be used instead. not working in kaggle.

def stemming(text):
    stemmer = PorterStemmer()
    text = [stemmer.stem(word) for word in text.split(" ")]
    return " ".join(text)

df['comment'] = df['comment'].apply(stemming)
# make X and y

X = df.drop(columns='sentiment')
y = df['sentiment']

X
# train test split the data

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=True,stratify=y)

print("The shape of X_train is",X_train.shape)
print("The shape of X_test is",X_test.shape)
```

```
# plot the two graphs

fig = plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
y_train.value_counts().plot(kind='bar')
plt.title("Train Data")

plt.subplot(1,2,2)
y_test.value_counts().plot(kind='bar')
plt.title("Test Data")
# transform the output variable
le = LabelEncoder()

y_train = le.fit_transform(y_train)
y_test = le.transform(y_test)
# form the preprocessor

preprocessor = ColumnTransformer(transformers=[
    ('encode',CountVectorizer(decode_error='ignore'),'comment'),
    ('scale',StandardScaler(),['word_count'])
],n_jobs=-1)

preprocessor
preprocessor.fit_transform(X_train.sample(20)).toarray()
results = dict(search.cv_results_)
# This line ONLY affects file saving, not model output
pd.DataFrame(results).to_csv("results.csv", index=False)

import pickle as pkl
```

```
with open("results.pkl", 'wb') as res:
  pkl.dump(results, res)

# best model

best_model = search.best_estimator_
import os
import joblib

# Save in a local "models" folder
output_dir = 'models'
os.makedirs(output_dir, exist_ok=True)

joblib.dump(best_model, os.path.join(output_dir, "best_model.joblib"))

print("The train accuracy score is: ",accuracy_score(y_train,y_pred_train))
print("The train f1 score is: ",f1_score(y_train,y_pred_train,average='macro'))

print("#"*50)

print("The test accuracy score is: ",accuracy_score(y_test,y_pred_test))
print("The test f1 score is: ",f1_score(y_test,y_pred_test,average='macro'))
print(f"The avg cross val score is {score.mean():.4f}")
def preprocess_manual_comment(comment):
    # Clean and process the input just like training
    comment = comment.lower()
    comment = ''.join([i for i in comment if not i.isdigit()])
    comment = re.sub('[%s]' % re.escape("""!"#$%&'()*+,‹-./:;<=>؟?@[\]^_`{|}~"""), ' ', comment)
    comment = comment.replace('؛',"")
    comment = re.sub(r'https?://\S+|www\.\S+', '', comment)
```

```
 # Remove stopwords
    stop_words = set(stopwords.words('english'))
    tokens = word_tokenize(comment)
    filtered = [w for w in tokens if w not in stop_words]

    # Stemming
    stemmer = PorterStemmer()
    stemmed = [stemmer.stem(word) for word in filtered]
    processed_text = " ".join(stemmed)

    return processed_text

def predict_sentiment(comment):
    processed_comment = preprocess_manual_comment(comment)
    word_count = len(processed_comment.split())

    # Wrap in DataFrame to match model input
    input_df = pd.DataFrame({'comment': [processed_comment], 'word_count': [word_count]})

    # Predict
    prediction = best_model.predict(input_df)[0]

    # Map back to label
    label = le.inverse_transform([prediction])[0]

    # Convert to numerical sentiment
    sentiment_map = {'Positive': 1, 'Negative': -1, 'Neutral': 0}
    return sentiment_map.get(label, "Unknown"), label

def predict_manual_comment_sentiment(comment):
```

```
    processed = preprocess_manual_comment(comment)
    wc = len(processed.split())

    input_df = pd.DataFrame({'comment': [processed], 'word_count': [wc]})
  pred = best_model.predict(input_df)[0]  # This will be 0, 1, or 2

    # Reverse lookup based on LabelEncoder order: [-1, 0, 1]
    label_mapping = {
        0: "Negative",   # encoded -1
        1: "Neutral",    # encoded  0
        2: "Positive"    # encoded  1
    }
    score_mapping = {
        0: -1,
        1: 0,
        2: 1
    }

    return label_mapping.get(pred, "Unknown"), score_mapping.get(pred, "Unknown")

print(predict_manual_comment_sentiment("This video is trash."))
print(predict_manual_comment_sentiment("Wow I loved every second of it!"))
print(predict_manual_comment_sentiment("It's just okay I guess."))
```

ENSENMBLE MODEL

(Combination of RandomForest, LR and SVR) –

```
!pip install transformers datasets
from google.colab import files
uploaded = files.upload()
# ✅ Disable WandB logs in Colab
os.environ["WANDB_DISABLED"] = "true"

# ✅ Load and clean dataset from Colab path
df = pd.read_csv("/content/Twitter_Data.csv.csv")
print("Available columns:", df.columns.tolist())
# Adjust column names below as per your dataset
# For example, use 'category' if it exists; else use 'sentiment' or 'label'
df = df[['text', 'sentiment']].dropna()
df.rename(columns={'text': 'clean_comment', 'sentiment': 'category'}, inplace=True)
df.rename(columns={'text': 'clean_comment'}, inplace=True)

# ✅ Train/test split for classic ML
X_train, X_test, y_train, y_test = train_test_split(df['clean_comment'], df['category'], test_size=0.2, random_state=42)
vectorizer = TfidfVectorizer(max_features=6000, ngram_range=(1,2))
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# ✅ Logistic Regression
logistic_model = LogisticRegression(C=5.0, max_iter=1000, solver='liblinear')
logistic_model.fit(X_train_vec, y_train)
y_pred = logistic_model.predict(X_test_vec)
print("\nLogistic Regression Performance:")
```

```
print(classification_report(y_test, y_pred))

# ✅ Random Forest Classifier
rf_model = RandomForestClassifier(n_estimators=300, max_depth=50, random_state=42)
rf_model.fit(X_train_vec, y_train)
y_pred_rf = rf_model.predict(X_test_vec)
print("\nRandom Forest Performance:")
print(classification_report(y_test, y_pred_rf))

# ✅ Inference using Logistic Regression
def predict_sentiment_logistic(text):
    vec = vectorizer.transform([text])
    return logistic_model.predict(vec)[0]

print("Predicted Sentiment (Logistic):", predict_sentiment_logistic("Buddhism is very peaceful and calming"))

# ✅ Label Mapping for RoBERTa
label_map = {-1: 0, 0: 1, 1: 2}
reverse_map = {0: -1, 1: 0, 2: 1}
df['label'] = df['category'].map(label_map)

# ✅ Convert to HuggingFace Dataset
hf_dataset = Dataset.from_pandas(df[['clean_comment', 'label']])
hf_dataset = hf_dataset.train_test_split(test_size=0.2)

# ✅ Tokenization for RoBERTa
tokenizer = RobertaTokenizer.from_pretrained("roberta-base")

def tokenize(example):
```

```
    return tokenizer(example["clean_comment"], truncation=True, padding="max_length", max_length=128)

tokenized_dataset = hf_dataset.map(tokenize, batched=True)

# ✅ Load RoBERTa model and set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = RobertaForSequenceClassification.from_pretrained("roberta-base", num_labels=3)
model.to(device)

# ✅ Metrics for Trainer
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='weighted', zero_division=0)
    acc = accuracy_score(labels, preds)
    return {"accuracy": acc, "f1": f1, "precision": precision, "recall": recall}

# ✅ Training Arguments
training_args = TrainingArguments(
    output_dir="./roberta_results",
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=6,
    weight_decay=0.01,
    learning_rate=2e-5,
    warmup_steps=500,
    logging_dir="./logs"
)
```

```
# ✅ Trainer Setup
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["test"],
    compute_metrics=compute_metrics
)

# ✅ Train the RoBERTa model
trainer.train()

# ✅ Evaluate the RoBERTa model
metrics = trainer.evaluate()
print("📊 RoBERTa Evaluation Metrics:")
roberta_results = {}
for k, v in metrics.items():
    if 'accuracy' in k or 'f1' in k or 'precision' in k or 'recall' in k:
        print(f"{k.capitalize()}: {v:.4f}")
        roberta_results[k] = v

# ✅ Summary comparison table
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

log_acc = accuracy_score(y_test, y_pred)
log_prec, log_rec, log_f1, _ = precision_recall_fscore_support(y_test, y_pred, average='weighted', zero_division=0)

rf_acc = accuracy_score(y_test, y_pred_rf)
rf_prec, rf_rec, rf_f1, _ = precision_recall_fscore_support(y_test, y_pred_rf, average='weighted', zero_division=0)
```

```
print(" 📋 Accuracy Comparison (Weighted Avg):")
print(f"Logistic Regression -> Accuracy: {log_acc:.4f}, Precision: {log_prec:.4f}, Recall: {log_rec:.4f}, F1: {log_f1:.4f}")
print(f"Random Forest -> Accuracy: {rf_acc:.4f}, Precision: {rf_prec:.4f}, Recall: {rf_rec:.4f}, F1: {rf_f1:.4f}")
print(f"RoBERTa -> Accuracy: {roberta_results['eval_accuracy']:.4f}, Precision: {roberta_results['eval_precision']:.4f}, Recall: {roberta_results['eval_recall']:.4f}, F1: {roberta_results['eval_f1']:.4f}")


# ✅ RoBERTa Inference Function
def predict_sentiment_roberta(text):
    model.eval()
    inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True, max_length=128)
    inputs = {k: v.to(device) for k, v in inputs.items()}
    with torch.no_grad():
        outputs = model(**inputs)
    predicted_class = torch.argmax(outputs.logits, dim=1).item()
    return reverse_map[predicted_class]


# ✅ Test RoBERTa Prediction
sample = "I love the peaceful teachings of Buddhism."
print("Predicted Sentiment (RoBERTa):", predict_sentiment_roberta(sample))
```

SVR Model –

```
from google.colab import files
uploaded = files.upload()
!ls
import io
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
from imblearn.over_sampling import RandomOverSampler
from imblearn.pipeline import Pipeline
from sklearn.svm import LinearSVC
df = pd.read_csv(io.StringIO(uploaded['Reddit_Data.csv'].decode('utf-8')))

# Display available columns and a data sample.
print("Columns in the dataset:", df.columns.tolist())
print("Dataset sample:")
print(df.head())

df['clean_comment'] = df['clean_comment'].fillna('')
# Also drop rows with missing labels if any.
df = df.dropna(subset=['category'])
X = df['clean_comment']
y = df['category']
print("\nLabel distribution after cleaning:")
print(y.value_counts())
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42, stratify=y
)
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(ngram_range=(1,2), sublinear_tf=True, max_features=10000)),
    ('oversample', RandomOverSampler(random_state=42)),
    ('clf', LinearSVC(class_weight='balanced', random_state=42))
])
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Negative', 'Neutral', 'Positive'],
            yticklabels=['Negative', 'Neutral', 'Positive'])
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.show()

# Step 9: Manual Prediction Section
def predict_manual_input(model_pipeline):
    """
    Repeatedly accepts manual text input and outputs the predicted label.
    Press ENTER without input to exit.
    """
    print("\nEnter manual text for prediction (press ENTER without input to exit):")
    while True:
```

```
        user_input = input("Input text: ").strip()
        if user_input == "":
            break
        prediction = model_pipeline.predict([user_input])
        print("Predicted label:", prediction[0])

# Run the manual prediction loop.
predict_manual_input(pipeline)
```

BERT Model –

```
!pip install transformers datasets torch scikit-learn pandas
!pip install transformers datasets -q
# ✅ STEP 2: Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

# ✅ STEP 3: Load and clean dataset
df = pd.read_csv("/content/Reddit_Data.csv")  # Path to your file
df = df[['clean_comment', 'category']].dropna()

# ✅ STEP 4: Train/test split
X_train, X_test, y_train, y_test = train_test_split(df['clean_comment'], df['category'], test_size=0.2, random_state=42)

# ✅ STEP 5: TF-IDF Vectorization
vectorizer = TfidfVectorizer(max_features=5000)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# ✅ STEP 6: Train Logistic Regression model
model = LogisticRegression()
model.fit(X_train_vec, y_train)

# ✅ STEP 7: Evaluation
y_pred = model.predict(X_test_vec)
```

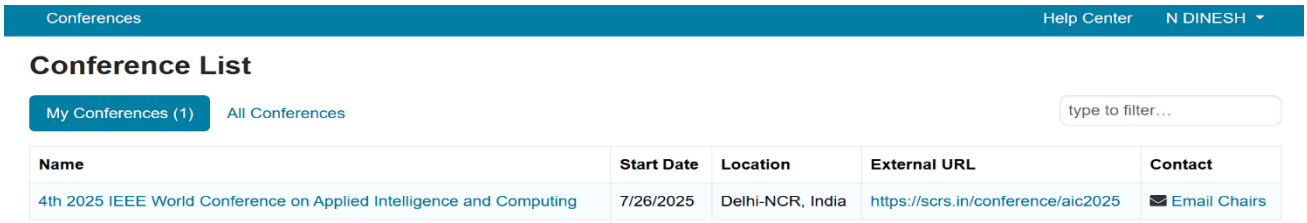
```
print(classification_report(y_test, y_pred))

# ✅ STEP 8: Inference function
def predict_sentiment(text):
    vec = vectorizer.transform([text])
    return model.predict(vec)[0]

# Example:
print("Predicted Sentiment:", predict_sentiment("Buddhism is very peaceful and calming"))
# ✅ Step 2: Imports
import pandas as pd
import numpy as np
import torch
import os

from datasets import Dataset
from transformers import BertTokenizer, BertForSequenceClassification, TrainingArguments, Trainer
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

# ✅ Step 3: Disable WandB (optional, prevents login issues)
os.environ["WANDB_DISABLED"] = "true"

# ✅ Step 4: Load your dataset
df = pd.read_csv("/content/Reddit_Data.csv")  # Use your CSV path
df = df[['clean_comment', 'category']].dropna()

# ✅ Step 5: Label mapping for 3-class classification
label_map = {-1: 0, 0: 1, 1: 2}
reverse_map = {0: -1, 1: 0, 2: 1}
df['label'] = df['category'].map(label_map)
```

```
# ✅ Step 6: Convert to Hugging Face Dataset
dataset = Dataset.from_pandas(df[['clean_comment', 'label']])
dataset = dataset.train_test_split(test_size=0.2)

# ✅ Step 7: Tokenizer
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

def tokenize(example):
    return tokenizer(example["clean_comment"], truncation=True, padding="max_length", max_length=128)

tokenized_dataset = dataset.map(tokenize, batched=True)

# ✅ Step 8: Load BERT model with 3 output labels
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=3)

# ✅ Step 9: Define evaluation metrics
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='weighted', zero_division=0)
    acc = accuracy_score(labels, preds)
    return {"accuracy": acc, "f1": f1, "precision": precision, "recall": recall}

# ✅ Step 10: Training arguments
training_args = TrainingArguments(
    output_dir="./bert_results",
    evaluation_strategy="epoch",
    save_strategy="epoch",
```

```
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=3,
    weight_decay=0.01,
    logging_dir="./logs",
    load_best_model_at_end=True,
    metric_for_best_model="f1"
)

# ✅ Step 11: Initialize Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["test"],
    tokenizer=tokenizer,
    compute_metrics=compute_metrics
)

# ✅ Step 12: Train the model
trainer.train()

# ✅ Step 13: Predict function
def predict_sentiment(text):

# ✅ Step 14: Test prediction
sample = "I love the peaceful teachings of Buddhism."
print("Predicted Sentiment:", predict_sentiment(sample))  # Should print -1, 0, or 1
```

APPENDIX B

CONFERENCE PRESENTATION

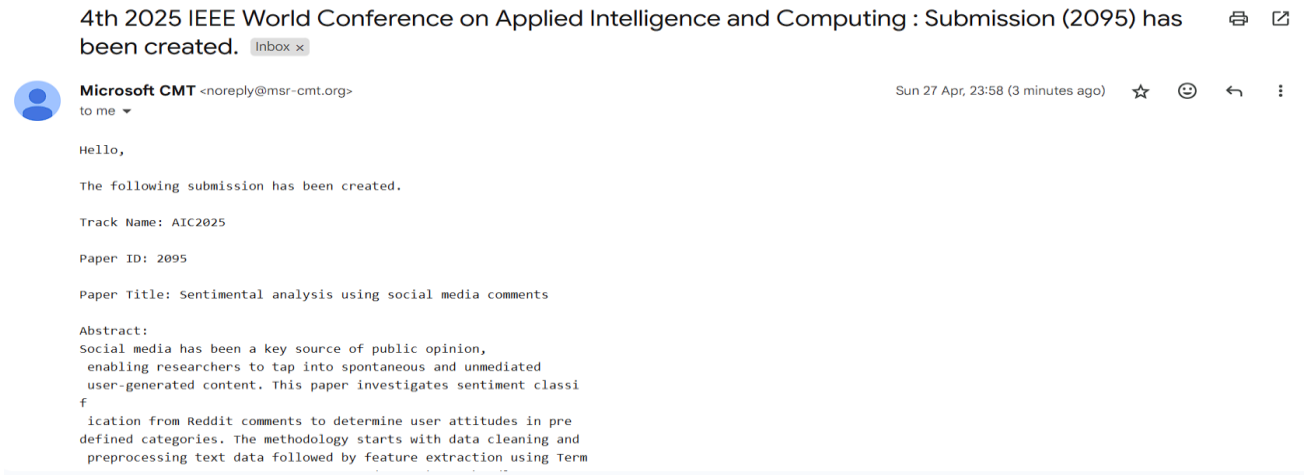


Conferences Help Center N DINESH

Conference List

My Conferences (1) All Conferences type to filter…

Name	Start Date	Location	External URL	Contact
4th 2025 IEEE World Conference on Applied Intelligence and Computing	7/26/2025	Delhi-NCR, India	https://scrs.in/conference/aic2025	Email Chairs



4th 2025 IEEE World Conference on Applied Intelligence and Computing : Submission (2095) has been created. Inbox ×

Microsoft CMT <noreply@msr-cmt.org> Sun 27 Apr, 23:58 (3 minutes ago)
to me

Hello,

The following submission has been created.

Track Name: AIC2025

Paper ID: 2095

Paper Title: Sentimental analysis using social media comments

Abstract:
Social media has been a key source of public opinion, enabling researchers to tap into spontaneous and unmediated user-generated content. This paper investigates sentiment classification from Reddit comments to determine user attitudes in predefined categories. The methodology starts with data cleaning and preprocessing text data followed by feature extraction using Term Frequency-Inverse Document Frequency (TF-IDF). To handle class imbalance—a typical problem in real-world datasets—we employ oversampling using the RandomOverSampler algorithm. Preprocessed data is then used to train a Linear Support Vector Classifier (LinearSVC), which is selected based on its ability to cope with high-dimensional text features. Model assessment is done using accuracy, confusion matrices, and classification reports. Results show that the employment of TF-IDF combined with oversampling has a significant impact on enhancing model performance, particularly in skewed data environments. This paper presents a practical, scalable approach of performing sentiment analysis on social media data.

Created on: Sun, 27 Apr 2025 18:28:40 GMT

Last Modified: Sun, 27 Apr 2025 18:28:40 GMT

Authors:
- nsdinesh1318@gmail.com (Primary)
- as0229@srmist.edu.in
- muralim@srmist.edu.in

Primary Subject Area: Computational intelligence

Secondary Subject Areas:
Intelligent Systems (Hardware & Software)

APPENDIX C
PUBLICATION DETAILS

APPENDIX D

PLAGIARISM REPORT

Murali M

Dinesh project

ASSIGNMENT 11

Cloud 2023-24

SRM Institute of Science & Technology

Document Details

Submission ID
trn:oid:::1:3230126283

Submission Date
Apr 27, 2025, 5:56 PM GMT+5:30

Download Date
Apr 27, 2025, 5:59 PM GMT+5:30

File Name
Report_Body_1_final.docx

File Size
2.0 MB

55 Pages

13,486 Words

85,771 Characters

turnitin Page 1 of 63 - Cover Page Submission ID trn:oid:::1:3230126283

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

- 118 Not Cited or Quoted 9%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

- 5% Internet sources
- 5% Publications
- 3% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.